



Module Code	:	CT038-3-2-OODJ – Object Oriented Development with Java
Intake Code	:	APD2F2509CS(CYB), APD2F2509CS(DA)
Lecturer Name	:	Lee Kim Keong
Hand in Date	:	1 st October 2025
Hand out Date	:	13 th December 2025
Group No.	:	Group J
Group Leader	:	Ahmed Mirahusain Alvi
Coursework Title	:	Course Recovery System (CRS) For Educational Institutions

STUDENT ID	STUDENT NAME
TP084807	AHMED MIRAHUSAIN ALVI
TP082459	MOHAMED ABDIFATAH ALI
TP085327	SULTAN ABDULLA OMAR TAKRORI
TP085061	MUHAMMAD TAIMOOR JAMAL
TP084630	MUHAMMAD ZIAD

TABLE OF CONTENTS

ACKNOWLEDGEMENT	1
CHAPTER 1: INTRODUCTION	2
1.1 Course Recovery System Overview	2
1.2 Assumptions	3
CHAPTER 2: DESIGN SOLUTION.....	6
2.1 Use Case Diagram.....	6
2.2 Use Case Diagram Description	7
2.2.1 Authenticate User	7
2.2.2 Manage User Accounts	8
2.2.3 Manage Password	9
2.2.4 Manage Failed Course Components	10
2.2.5 Manage Recovery Recommendations.....	11
2.2.6 Manage Recovery Milestones.....	12
2.2.7 Track Recovery Progress	13
2.2.8 Check Progression Eligibility	14
2.2.9 Manage Student Enrollment	15
2.2.10 Generate Academic Report	16
2.2.11 Send Email Notification.....	17
2.2.12 Log Timestamp to Binary File	18
2.2.13 Validate User Credentials.....	19
2.2.14. Send Password Recovery Email	20
2.2.15 Send Enrollment Confirmation Email	21
2.2.16 Calculate Semester GPA	22

2.3 Class Diagram	23
CHAPTER 3: PROGRAM IMPLEMENTATION	24
3.1 User Management	24
3.1.1 Add User	24
3.1.2 Edit User	30
3.1.3 Delete User.....	32
3.1.4 Change Password	32
3.1.5 User Activity Log.....	37
3.2 Eligibility Check and Enrolment	38
3.2.1 Main Graphical User Interface.....	38
3.2.3 Approving ineligible and eligible students	41
3.2.4 Denying Enrolment for Eligible and Ineligible Students.....	46
3.2.5 Back button	48
3.3 Academic Performance Report	49
3.3.1 Main GUI Components.....	49
3.3.2 System Flow.....	51
3.4 Course Recovery Plan.....	59
3.4.1 Filtered Class Test View.....	59
3.4.2 Component Filtering	60
3.4.3 Filtered Class Test View.....	61
3.4.4 Student Status Validation	62
3.4.5 Mandatory Selection Validation.....	63
3.4.6 Recommendation Through Email	64
3.5 Email Notification Service.....	65
3.5.1 Main Interface	65

3.5.2 Email Notification Execution	68
3.5.3 Invalid Email Handling.....	71
3.5.4 Clear Button Behavior	72
3.5.5 Save Draft Functionality	73
3.5.6 Load Draft Functionality.....	75
3.5.7 Send Notification button.....	78
3.5.8 Sent Email History & Audit Log	79
CHAPTER 4: OBJECT-ORIENTED PROGRAMMING CONCEPTS.....	82
4.1 User Management	82
4.1.1 Abstraction	82
4.1.2 Encapsulation.....	83
4.1.3 Polymorphism.....	84
4.1.4 Separation of Concerns	85
4.2 Eligibility Check and Enrolment	86
4.2.1 Abstraction	86
4.2.2 Inheritance.....	87
4.2.3 Encapsulation.....	88
4.2.4 Interface	89
4.2.5 Polymorphism.....	90
4.3 Academic Performance Reporting	92
4.3.1 Encapsulation.....	92
4.3.2 Abstraction	93
4.4 Course Recovery Plan.....	95
4.4.1 Abstraction	95
4.4.2 Encapsulation.....	98

4.5 Email Notification Service.....	100
4.5.1 Encapsulation.....	100
4.5.2 Inheritance.....	102
4.5.3 Polymorphism.....	103
4.5.4 Abstraction	106
CONCLUSION.....	112
REFERENCES	113
WORKLOAD MATRIX.....	115

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to **Mr. Lee Kim Keong** for his valuable guidance, support, and encouragement throughout the preparation and completion of our **Object-Oriented Development with Java (OODJ)** assignment. His insights, clear explanations, and constructive feedback greatly helped us in understanding the core concepts and applying them effectively in our work. This assignment not only enhanced our technical knowledge but also improved our analytical and problem-solving skills. We are sincerely grateful for the opportunity to learn under his supervision and for the continuous support he has shown us. We also express our deep gratitude to our colleagues and peers, whose collaboration and exchange of ideas have greatly enriched our understanding of the subject. A heartfelt thank you to our friends: Mohamed Abdifatah Ali, Sultan Abdulla Omar Takrori, Ahmed Mirahusain Alvi, Muhammad Taimoor Jamal and Muhammad Ziad for their continuous support, motivation, and cooperation throughout this journey.

CHAPTER 1: INTRODUCTION

1.1 Course Recovery System Overview

This Course Recovery System (CRS) is specifically designed to streamline and optimize the academic recovery workflow of educational institutions, enabling students who have failed or received incomplete grades to regain academic standing without delaying graduation. As educational institutions scale their operations, managing course recovery processes manually becomes increasingly inefficient, error-prone, and challenging to track. To address these operational difficulties, a comprehensive Java application grounded in Object-Oriented Programming (OOP) principles was developed.

The system accommodates multiple user roles such as Academic Officers and Course Administrators, each with distinct access privileges and responsibilities. Key functionalities include user authentication and authorization with role-based access control, course recovery plan management with milestone tracking and progress monitoring, eligibility verification based on CGPA calculations and number of failed courses, academic performance reporting with PDF export capabilities, and automated email notifications for critical system events such as password recovery, recovery plan updates, and performance reports.

The primary objective of this initiative is to implement core OOP concepts within an academic management context, ensuring that the resulting codebase is modular, maintainable, and scalable. The system architecture reflects real-world relationships between academic entities while enforcing role-based access control to maintain organized and secure operations throughout the course recovery lifecycle. By encapsulating business logic, abstracting complex workflows, and leveraging polymorphism for flexible component design, the system demonstrates professional software engineering practices aligned with industry standards.

This document elaborates on the system's OOP implementations, architectural design decisions, user interface components, and overall functionality. It includes comprehensive use case diagrams with detailed descriptions, class diagrams illustrating entity relationships and OOP principal applications, program output screenshots with appropriate explanations, system limitations, and recommendations for future enhancements

1.2 Assumptions

To ensure the successful development and functionality of the Course Recovery System (CRS), the following assumptions have been made during the project:

1. Single-User Login at a Time

The system assumes one user will interact with the application at a time. Multi-user concurrency and simultaneous session management are not implemented in this version.

2. All Users Are Pre-Registered by the Administrator

Only authorized personnel (such as the Academic Officer or Course Administrator) can create and manage user accounts. Students and instructors cannot self-register. All user credentials must be provisioned through the administrative interface.

3. Unique Identifiers Are Maintained

Every entity within the system—including users, courses, students, recovery plans, and academic reports—is uniquely identified using auto-generated IDs to prevent duplication and maintain data integrity across all operations.

4. File-Based Storage Only

The system uses text files for data persistence, not relational databases. All data storage, retrieval, and updates are performed via file handling mechanisms, ensuring compliance with the project's technical requirements.

5. Valid Input Format and User Validation

The system assumes users will input data in the correct format (e.g., numeric quantities, valid date formats, proper email addresses). Basic validation is implemented to ensure data consistency and prevent logical errors.

6. CGPA Calculation Based on Credit Hour System

Academic eligibility is computed using the standard Credit Hour System where $CGPA = (Total\ Grade\ Points\ of\ all\ courses\ taken) / (Total\ Credit\ Hours\ of\ all\ courses\ taken)$. The system follows the 4.0 grading scale as specified in the coursework requirements.

7. Eligibility Criteria for Progression

Students must meet two mandatory criteria to progress to the next level of study:

- Maintain a minimum CGPA of 2.0
- Have no more than three failed courses

Students who do not meet these criteria are flagged as ineligible and must undergo the course recovery process.

8. Course Recovery Attempt Policy

Each course has three attempt opportunities to pass:

- 1st attempt: Initial enrollment
- 2nd attempt: Failed component only (resubmission or resit)
- 3rd attempt: All assessment components must be retaken

This policy is enforced throughout the recovery planning and tracking modules.

9. No Network or Internet Requirement for Core Functionality

The application is developed as a standalone desktop system and does not require internet access for its core operations. However, the email notification feature requires network connectivity to send automated messages via the Java Mail API.

10. Authorized Access Only

Users are expected to log in using their assigned credentials. Login and logout timestamps are logged in binary form for audit purposes. Unauthorized access attempts are not simulated or handled beyond basic authentication failure messages.

11. Predefined Course-Instructor Relationships

Each course must be linked to a valid, pre-registered instructor. Courses cannot exist in the system without an associated instructor responsible for evaluating recovery progress.

12. Third-Party API Integration

The system integrates with external libraries for specific functionalities:

- Java Mail API for email notification services
- Text PDF for generating and exporting academic performance reports

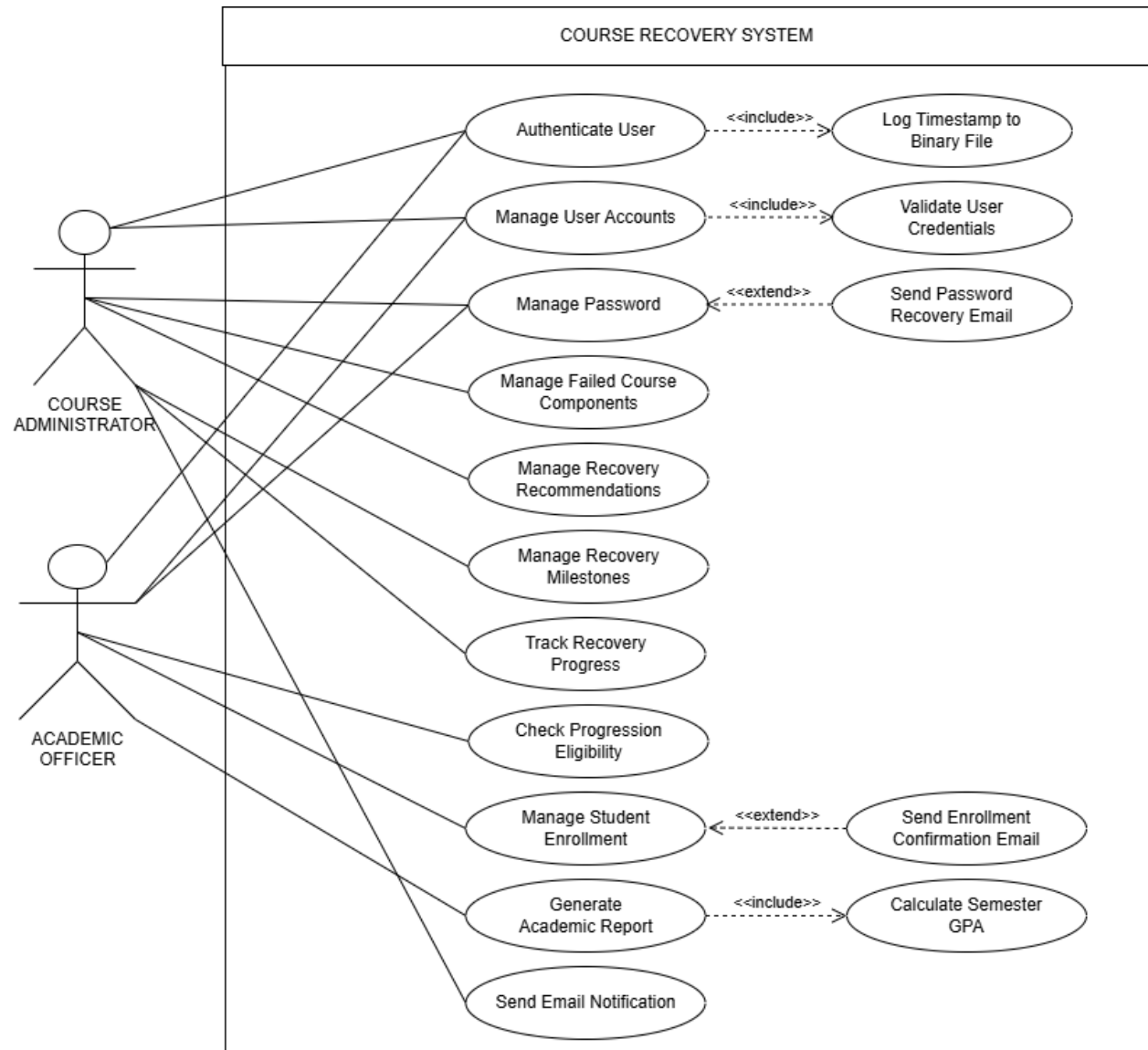
These dependencies are assumed to be correctly configured and available at runtime.

13. GUI-Driven Interface

The system is designed with a graphical user interface (GUI) that provides menu-driven options for all user roles. Command-line or text-based interfaces are not provided.

CHAPTER 2: DESIGN SOLUTION

2.1 Use Case Diagram



2.2 Use Case Diagram Description

2.2.1 Authenticate User

Use Case	Authenticate User
Brief Description	This use case allows a Course Administrator or Academic Officer to log into the Course Recovery System.
Actors	Course Administrator, Academic Officer
Preconditions	The actor has an active system account and access to the login interface.
Main Flow	(a) The use case begins when the actor selects the option to log into the system. (b) The system prompts the actor to enter their email and password (ref. Alternative flow). (c) The actor enters the required credentials into the login form. (d) The system validates the credentials against the user database (ref. Alternative flow). (e) The system logs the timestamp of the login attempt for audit purposes.
Alternative Flows	(a) One or more required fields are left empty. The system displays an error message instructing the actor to complete all mandatory fields before continuing. (b) The entered credentials do not match any registered account. The system notifies the actor of invalid credentials and requests a retry.
Postconditions	A valid session is created, and the login timestamp is recorded in the system log.

2.2.2 Manage User Accounts

Use Case	Manage User Accounts
Brief Description	This use case allows the Course Administrator to manage user account records.
Actors	Course Administrator
Preconditions	The Course Administrator has successfully authenticated and gained system access.
Main Flow	(a) The use case begins when the administrator selects the User Management module. (b) The system retrieves and displays a list of all registered user accounts. (c) The administrator selects an account to create, modify, or delete and enters the required details (ref. Alternative flow). (d) The system validates the submitted data for completeness and correctness (ref. Alternative flow). (e) The system updates the user account records accordingly and displays a confirmation message.
Alternative Flows	(a) The administrator provides incomplete or invalid data. The system prompts the administrator to correct the information before proceeding. (b) The system encounters a validation error. The administrator is notified and instructed to rectify the issue.
Postconditions	User account information is updated and stored in the system database.

2.2.3 Manage Password

Use Case	Manage Password
Brief Description	This case allows authenticated users to change their password or request a password reset.
Actors	Course Administrator, Academic Officer
Preconditions	The actor is authenticated or has initiated a password reset request.
Main Flow	(a) The use case begins when the actor selects the Password Management module. (b) The system prompts the actor to enter the required details such as current password, new password, or reset verification (ref. Alternative flow). (c) The system processes the password update or reset request and displays a confirmation.
Alternative Flows	(a) The actor selects “Forgot Password.” The system triggers the password recovery process and requests verification details. (b) The chosen password does not meet system policy requirements. The system displays a message indicating the password weakness and requests a stronger password.
Postconditions	The user’s password is successfully updated or reset in the system.

2.2.4 Manage Failed Course Components

Use Case	Manage Failed Course Components
Brief Description	This case allows the Course Administrator to update student failure records.
Actors	Course Administrator
Preconditions	Administrator authenticated.
Main Flow	(a) The use case begins when the administrator selects the Failed Course Components module. (b) The system retrieves and displays the list of students with failed components (ref. Alternative flow). (c) The administrator updates or adds failure details for a selected student. (d) The system saves the updated failure record and displays a confirmation message.
Alternative Flows	(a) No students with failed components are found. The system notifies the administrator that no records are available for update.
Postconditions	The failed course component records are updated and stored in the database.

2.2.5 Manage Recovery Recommendations

Use Case	Manage Recovery Recommendations
Brief Description	This use case allows the Course Administrator to create or update recovery recommendations for students.
Actors	Course Administrator
Preconditions	The student has at least one failed component recorded.
Main Flow	(a) The use case begins when the administrator opens the Recovery Recommendations module. (b) The system retrieves and displays existing recovery information for eligible students (ref. Alternative flow). (c) The administrator enters or updates recovery recommendations as required. (d) The system saves the recommendations and confirms the update.
Alternative Flows	(a) No eligible students are found. The system informs the administrator that there are no students requiring recovery recommendations.
Postconditions	Updated recovery recommendations are added to the student's academic record.

2.2.6 Manage Recovery Milestones

Use Case	Manage Recovery Milestones
Brief Description	This use case allows administrators to manage milestone plans for student recovery.
Actors	Course Administrator
Preconditions	A recovery plan already exists for the student.
Main Flow	a) The use case begins when the administrator accesses the Recovery Milestones module. (b) The system retrieves and displays milestones associated with the selected student's recovery plan (ref. Alternative flow). (c) The administrator updates or adds milestone entries. (d) The system stores the updated milestone data and displays a confirmation.
Alternative Flows	(a) No milestones exist for the selected student. The system notifies the administrator and provides an option to add new milestones.
Postconditions	Updated recovery recommendations are added to the student's academic record.

2.2.7 Track Recovery Progress

Use Case	Track Recovery Progress
Brief Description	Monitors student recovery milestone progress.
Actors	Course Administrator
Preconditions	Recovery milestones have been created.
Main Flow	(a) The use case begins when the administrator opens the Progress Tracking module. (b) The system retrieves and displays milestone completion percentages and student progress indicators (ref. Alternative flow). (c) The administrator reviews the displayed progress to assess student improvement.
Alternative Flows	(a) No progress records are available. The system notifies the administrator that no updates have been recorded.
Postconditions	The administrator has viewed the most recent recovery progress data.

2.2.8 Check Progression Eligibility

Use Case	Check Progression Eligibility
Brief Description	This use case allows an Academic Officer to determine whether a student meets the requirements to progress to the next academic level.
Actors	Academic Officer
Preconditions	The student's academic record exists in the system.
Main Flow	(a) The use case begins when the Academic Officer selects the Progression Eligibility module. (b) The system retrieves the student's complete academic history and displays relevant details (ref. Alternative flow). (c) The system applies academic progression rules and evaluates the student's eligibility. (d) The system displays a clear eligibility result, indicating whether the student may progress
Alternative Flows	(a) The system is unable to retrieve complete student data due to missing grades. The system alerts the Academic Officer and requests verification.
Postconditions	The student's progression eligibility status is generated and displayed to the Academic Officer.

2.2.9 Manage Student Enrollment

Use Case	Manage Student Enrollment
Brief Description	This case allows the Academic Officer to manage and update student enrollment information.
Actors	Academic Officer
Preconditions	Student record available.
Main Flow	(a) The use case begins when the Academic Officer accesses the Enrollment Management module. (b) The system loads and displays the student's enrollment details (ref. Alternative flow). (c) The Academic Officer updates the enrollment information by adding, modifying, or removing entries (ref. Alternative flow). (d) The system sends a confirmation email to the student regarding the updated enrollment status. (e) The system saves the revised enrollment information.
Alternative Flows	(a) The system cannot locate student records. The Academic Officer is notified to verify student details. (b) Required enrollment fields are incomplete. The system prompts the Academic Officer to complete all required information before saving.
Postconditions	The student's enrollment record is updated and stored in the system database.

2.2.10 Generate Academic Report

Use Case	Generate Academic Report
Brief Description	This use case allows the Academic Officer to generate an academic performance report for a student.
Actors	Academic Officer
Preconditions	The student's academic data is available in the system.
Main Flow	(a) The use case begins when the Academic Officer selects the Academic Report module. (b) The system retrieves the student's academic details including grades, credits, and completed courses (ref. Alternative flow). (c) The system calculates the semester GPA. (d) The system generates a structured academic report and displays it for review.
Alternative Flows	(a) One or more academic records are missing. The system notifies the officer that the report cannot be generated until the missing data is resolved.

2.2.11 Send Email Notification

Use Case	Send Email Notification
Brief Description	This use allows the Academic Officer to send system-generated notifications to users.
Actors	Academic Officer
Preconditions	Valid email addresses exist for the recipients
Main Flow	(a) The use case begins when the Academic Officer selects the Notification module. (b) The system prompts the officer to compose a message and choose recipients. (c) The system sends the email notification and displays a confirmation message (ref. Alternative flow).
Alternative Flows	(a) The email server encounters an error. The system alerts the Academic Officer and prompts them to retry or contact support.
Postconditions	The notification email is successfully sent to all intended recipients.

2.2.12 Log Timestamp to Binary File

Use Case	Log Timestamp to Binary File
Brief Description	This case allows the system to store login timestamps into a binary log file for audit purposes.
Actors	System
Preconditions	A login attempt is made by any authenticated actor.
Main Flow	(a) The use case begins when the system receives a login event. (b) The system captures and writes the timestamp to the binary log file (ref. Alternative flow). (c) The system confirms that the timestamp entry has been successfully recorded.
Alternative Flows	(a) The binary log file is unavailable or corrupted. The system registers failure and triggers a fallback logging mechanism.
Postconditions	(a) The timestamp is appended to the binary log file or stored via the fallback method.

2.2.13 Validate User Credentials

Use Case	Validate User Credentials
Brief Description	This use case allows the system to verify login credentials against stored user data.
Actors	System
Preconditions	The actor has submitted credentials through the login interface.
Main Flow	Main Flow (a) The use case begins when the system receives the email and password entered by the actor. (b) The system compares the submitted credentials with the stored records (ref. Alternative flow). (c) The system returns a result indicating whether authentication is successful or denied.
Alternative Flows	(a) The system cannot reach the authentication database. The system displays an error message informing the actor of temporary system unavailability.
Postconditions	The system returns a validation outcome to the login module.

2.2.14. Send Password Recovery Email

Use Case	Send Password Recovery Email
Brief Description	This use case allows the system to send a password reset link to the user requesting password recovery.
Actors	Course Administrator
Preconditions	A password reset request has been initiated by the user.
Main Flow	.(a) The use case begins when the system generates a unique recovery token for the user. (b) The system sends a password reset email containing the token (ref. Alternative flow). (c) The system displays a confirmation indicating the email has been sent.
Alternative Flows	(a)The email address provided does not match any registered account. The system notifies the user that the recovery email cannot be sent.
Postconditions	The user receives a valid password reset link in their email.

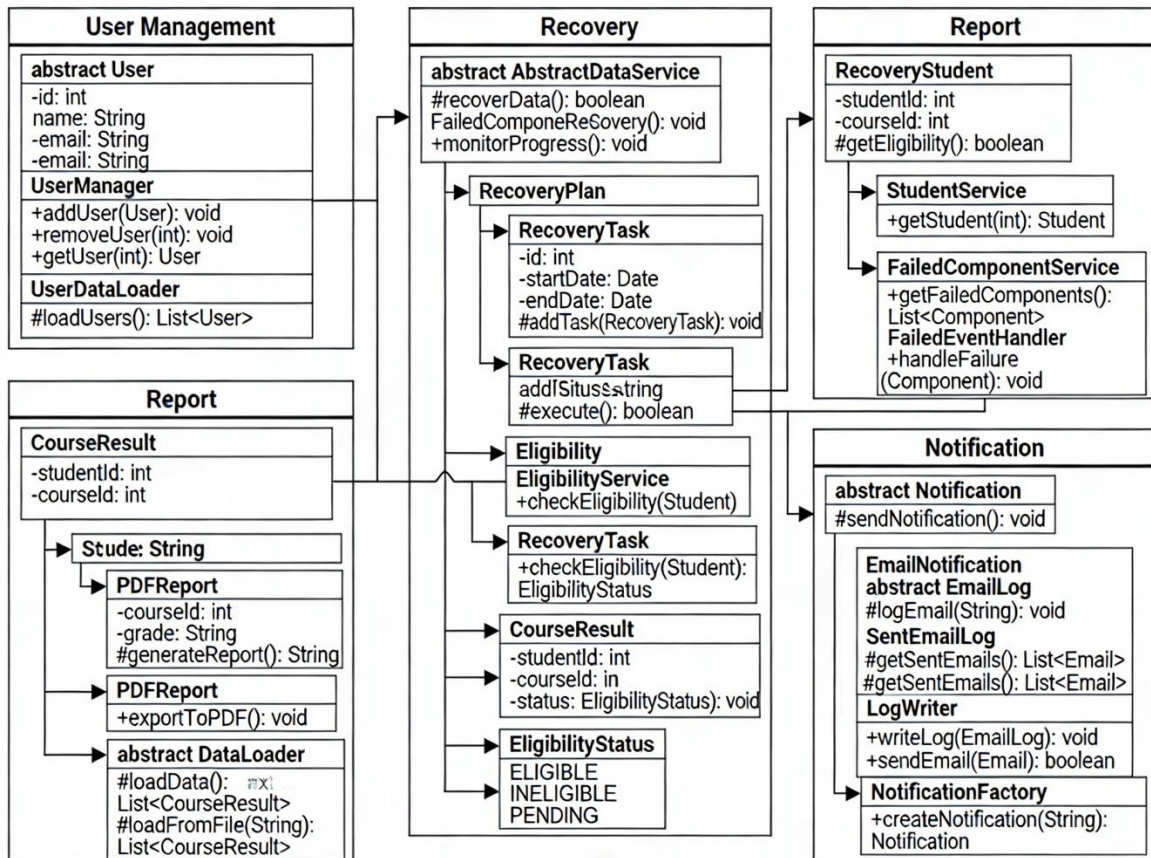
2.2.15 Send Enrollment Confirmation Email

Use Case	Send Enrollment Confirmation Email
Brief Description	This case allows the system to send an email to confirm that enrollment updates have been successfully processed.
Actors	Academic Officer
Preconditions	The student enrollment information has been modified.
Main Flow	(a) The use case begins when the system detects an enrollment change. (b) The Academic Officer generates a confirmation message summarizing the update. (c) The system sends the confirmation email to the student (ref. Alternative flow).
Alternative Flows	(a)The system fails to send email due to server issues. The system alerts the Academic Officer and logs the error.
Postconditions	The student receives confirmation of their updated enrollment status.

2.2.16 Calculate Semester GPA

Use Case	Calculate Semester GPA
Brief Description	This use case allows the system to compute a student's GPA based on stored grade data.
Actors	System
Preconditions	Grade data for the current semester exists.
Main Flow	(a) The use case begins when the system retrieves all course grades for the student. (b) The system applies the GPA calculation formula based on grade points and credits (ref. Alternative flow). (c) The system generates the final GPA and returns it to the requesting module.
Alternative Flows	(a) One or more grade entries are missing. The system cannot perform GPA calculation and notify the Academic Officer accordingly.
Postconditions	The calculated GPA is returned and made available for use in academic reports or progression checks.

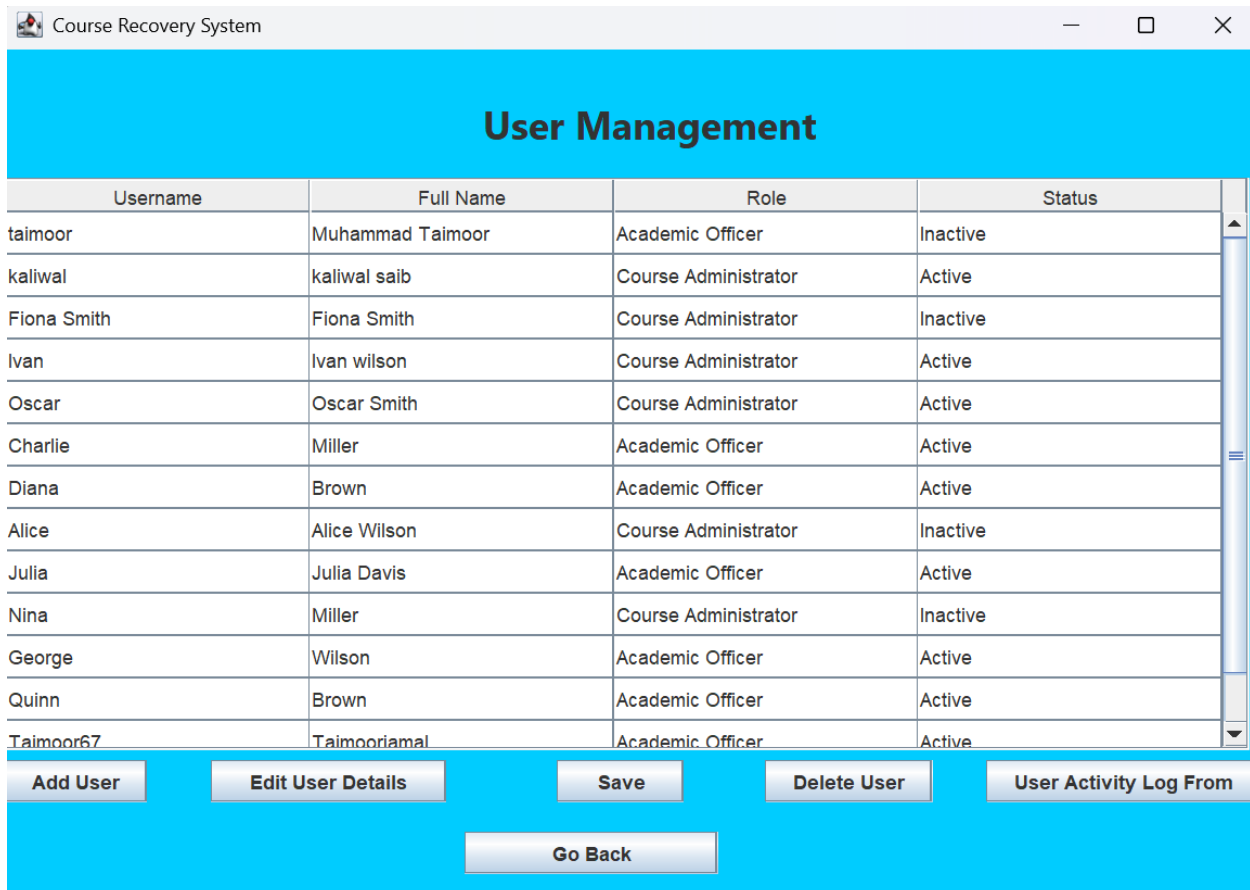
2.3 Class Diagram



CHAPTER 3: PROGRAM IMPLEMENTATION

3.1 User Management

User Management module is meant to enable the Course Administrator manage systematic users effectively and safely. The module offers the much-needed functionalities of the addition of new users, updating user details, updating passwords, and deactivation of user accounts upon the need arising. The records of all the users are presented in a JTable, which can be viewed and selected easily. File handling is used to store user data whereas the operations involving the passwords are kept separately to enhance security and clean system structure.



Username	Full Name	Role	Status
taimoor	Muhammad Taimoor	Academic Officer	Inactive
kaliwal	kaliwal saib	Course Administrator	Active
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan wilson	Course Administrator	Active
Oscar	Oscar Smith	Course Administrator	Active
Charlie	Miller	Academic Officer	Active
Diana	Brown	Academic Officer	Active
Alice	Alice Wilson	Course Administrator	Inactive
Julia	Julia Davis	Academic Officer	Active
Nina	Miller	Course Administrator	Inactive
George	Wilson	Academic Officer	Active
Quinn	Brown	Academic Officer	Active
Taimoor67	Taimooriamal	Academic Officer	Active

Buttons: Add User, Edit User Details, Save, Delete User, User Activity Log From, Go Back

3.1.1 Add User

Add User feature enables the administrator to add a new user by typing in user name, complete name, role and password. A confirm password field is added to make sure that the password is

typed correctly. The table does not show the passwords and it is handled by the system. On addition, the user data is stored in a text file and automatically reflected in the table.

Add User Form:

Add User interface prompting administrator to enter user details.

Username	Full Name	Role	Status
kailwal	kailwal saib	Course Administrator	Active
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan wilson	Course Administrator	Active
Oscar	Oscar Smith	Course Administrator	Active
Charlie	Miller	Academic Officer	Active
Diana	Brown	Academic Officer	Active
Alice	Alice Wilson	Course Administrator	Inactive
Julia	Julia Davis		Active
Nina	Miller		Inactive
George	Wilson		Active
Quinn	Brown		Active
Taimoor67	Taimoorjama		Active
MOHA	MOHA ABDIFATAH	Course Administrator	active

Input

Enter username:

Taimoor252002

OK Cancel

Add User Edit User Details Save Delete User User Activity Log From

Go Back

Course Recovery System

User Management

Username	Full Name	Role	Status
taimoor	Muhammad Taimoor	Academic Officer	Inactive
kaliwal	kaliwal saib	Course Administrator	Active
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan wilson		Active
Oscar	Oscar Smith		Active
Charlie	Miller		Active
Diana	Brown		Active
Alice	Alice Wilson		Inactive
Julia	Julia Davis	Academic Officer	Active
Nina	Miller	Course Administrator	Inactive
George	Wilson	Academic Officer	Active
Quinn	Brown	Academic Officer	Active
Taimoor67	Taimooriamal	Academic Officer	Active

Role Selection

×

Select a role:

Academic Officer

Academic Officer

Course Administrator

Add User

Edit User Details

Save

Delete User

User Activity Log From

Go Back

User Management

Username	Full Name	Role	Status
kaliwal	kaliwal saib	Course Administrator	Active
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan wilson	Course Administrator	Active
Oscar	Oscar Smith	Course Administrator	Active
Charlie	Miller	Academic Officer	Active
Diana	Brown	Academic Officer	Active
Alice	Alice Wilson	Course Administrator	Inactive
Julia	Julia		Active
Nina	Miller		Inactive
George	Wilso		Active
Quinn	Brow		Active
Taimoor67	Taimoor		Active
MOHA	MOHA ABDIFATAH	Course Administrator	active

Add User

[Edit User Details](#)

Save

Delete User

User Activity Log From

Go Back

Course Recovery System

User Management

Username	Full Name	Role	Status
kaliwai	kaliwai saib	Course Administrator	Active
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan wilson	Course Administrator	Active
Oscar	Oscar Smith	Course Administrator	Active
Charlie	Miller	Academic Officer	Active
Diana	Brown	Academic Officer	Active
Alice	Alice Wilson	Course Administrator	Inactive
Julia			
Nina			
George			
Quinn			
Taimoor67	Taimoorjama	Academic Officer	Active
MOHA	MOHA ABDIFATAH	Course Administrator	active

Weak Password

 Password must be at least 4 characters long. User registration cancelled.

OK

Add User Edit User Details Save Delete User User Activity Log From

Go Back

Weak Password Warning alert

 Course Recovery System

User Management

Username	Full Name	Role	Status
kaliwal	kaliwal saib	Course Administrator	Active
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan wilson	Course Administrator	Active
Oscar	Oscar Smith	Course Administrator	Active
Charlie	Miller	Academic Officer	Active
Diana	Brown	Academic Officer	Active
Alice	Alice Wilson	Course Administrator	Inactive
Julia	Julia D	Success	Active
Nina	Miller		Inactive
George	Wilson		Active
Quinn	Brown		Active
Taimoor67	Taimoorjama	Academic Officer	Active
MOHA	MOHA ABDIFATAH	Course Administrator	active


User 'Taimoor252002' registered successfully!

OK

Add User

Edit User Details

Save

Delete User

User Activity Log From

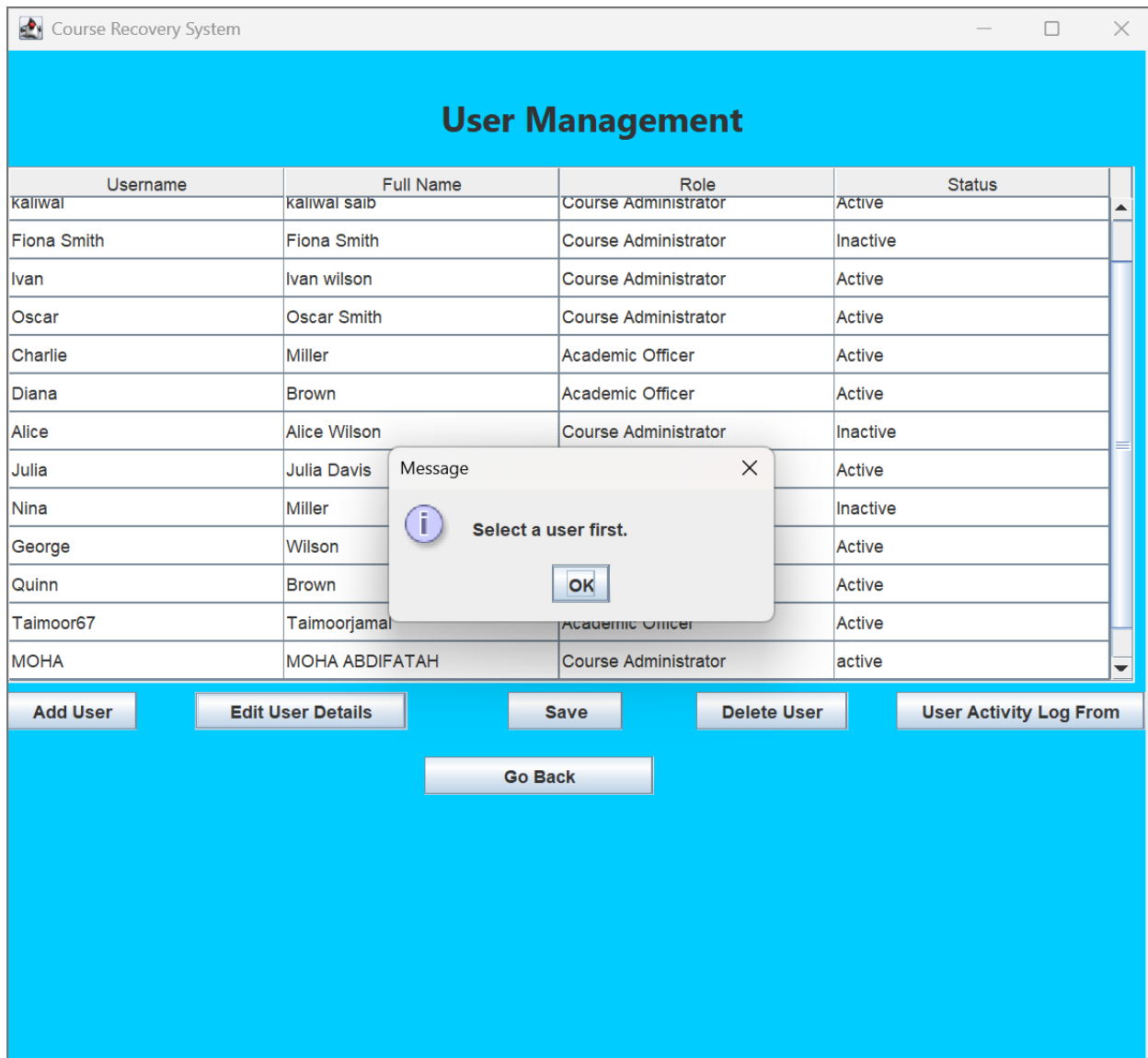
Go Back

User added Successfully!

3.1.2 Edit User

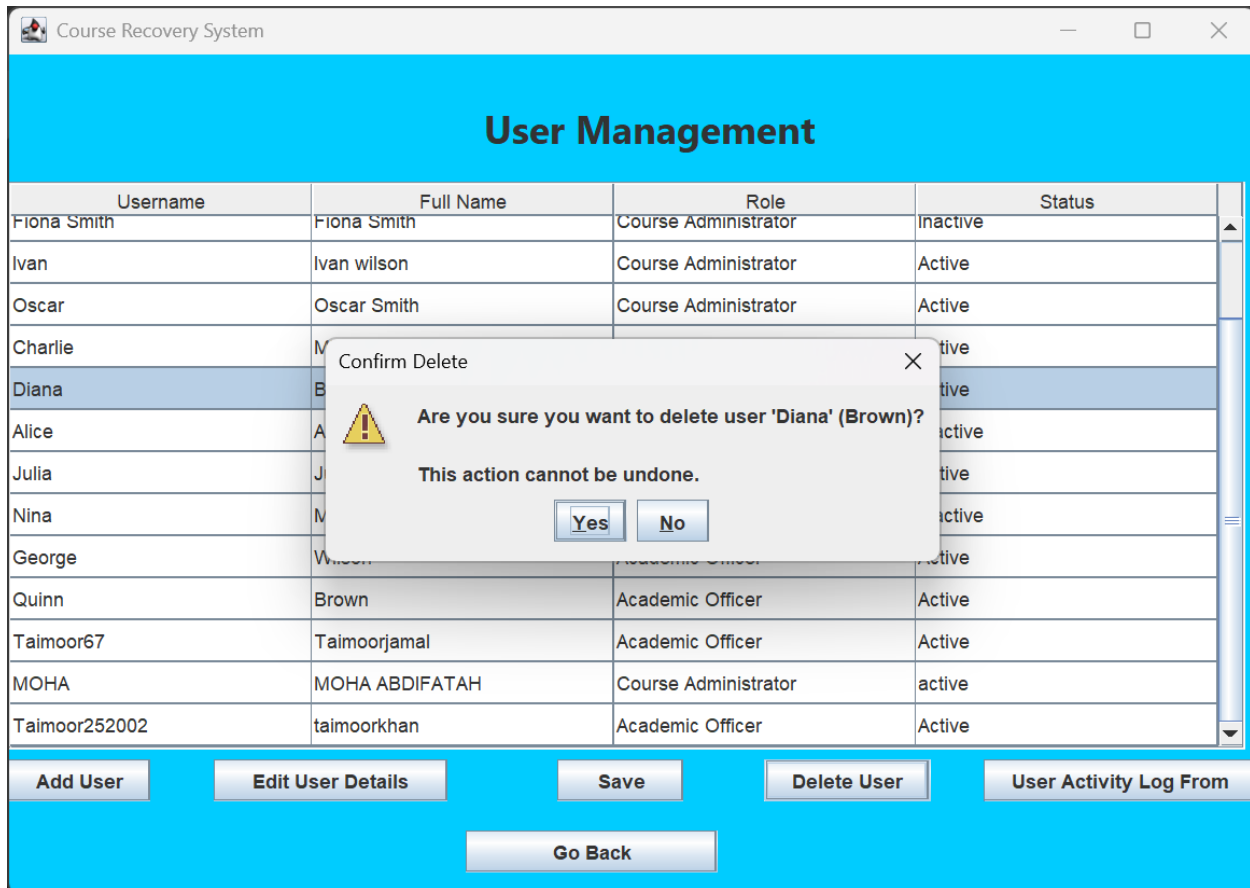
The Edit User option allows the administrator to edit the full name, role or account status of an existing user. The information of the selected user is entered in the input fields to edit it. The changes are authenticated by the system and then saved. Once the update is successful, the jtable will automatically update and show the current user information.

User selected



Edit User interface showing editable user details

3.1.3 Delete User



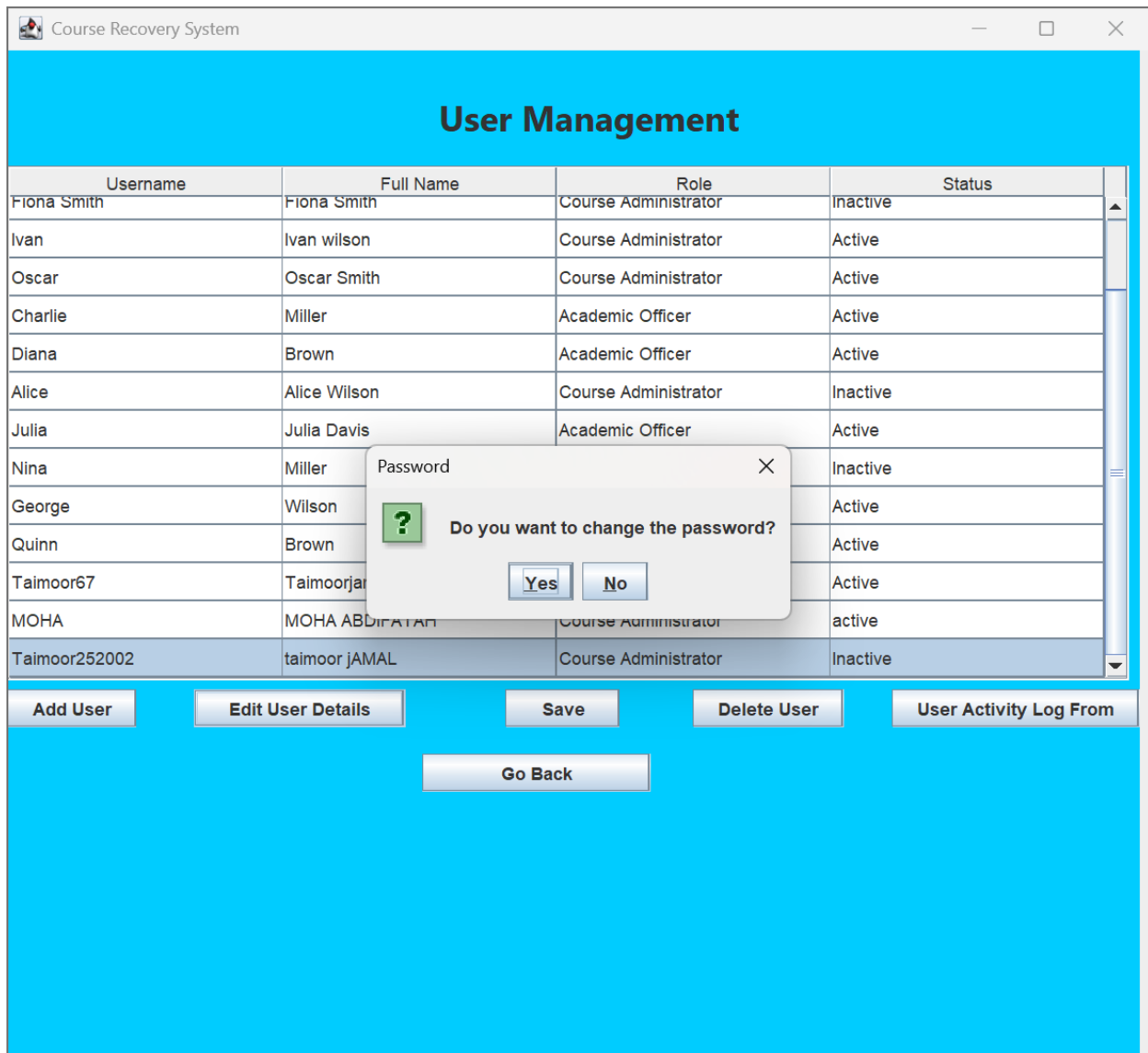
Confirmation dialog appears

confirmation dialog for deleting user

3.1.4 Change Password

Change Password option provides secure password changes whereby he/she is required to key in their previous password before they change it. A new password can only be entered and confirmed by the user after a successful verification. UserManager class is used to perform all password checks and updates. This division ensures that vulnerable logic is not exposed to the user interface and the overall security of the system is enhanced.

User can change the Password from Edit User Details



Course Recovery System

User Management

Username	Full Name	Role	Status
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan wilson	Course Administrator	Active
Oscar	Oscar Smith	Course Administrator	Active
Charlie	Miller	Academic Officer	Active
Diana	Brown	Academic Officer	Active
Alice	Alice Wilson	Course Administrator	Inactive
Julia	Julia Davis	Academic Officer	Active
Nina	Miller		Inactive
George	Wilson		Active
Quinn	Brown		Active
Taimoor67	Taimoorja		Active
MOHA	MOHA ABDI ATAN	Course Administrator	active
Taimoor252002	taimoor JAMAL	Course Administrator	Inactive

Password X
? Do you want to change the password?
Yes No

Add User Edit User Details Save Delete User User Activity Log From

Go Back

 Course Recovery System

User Management

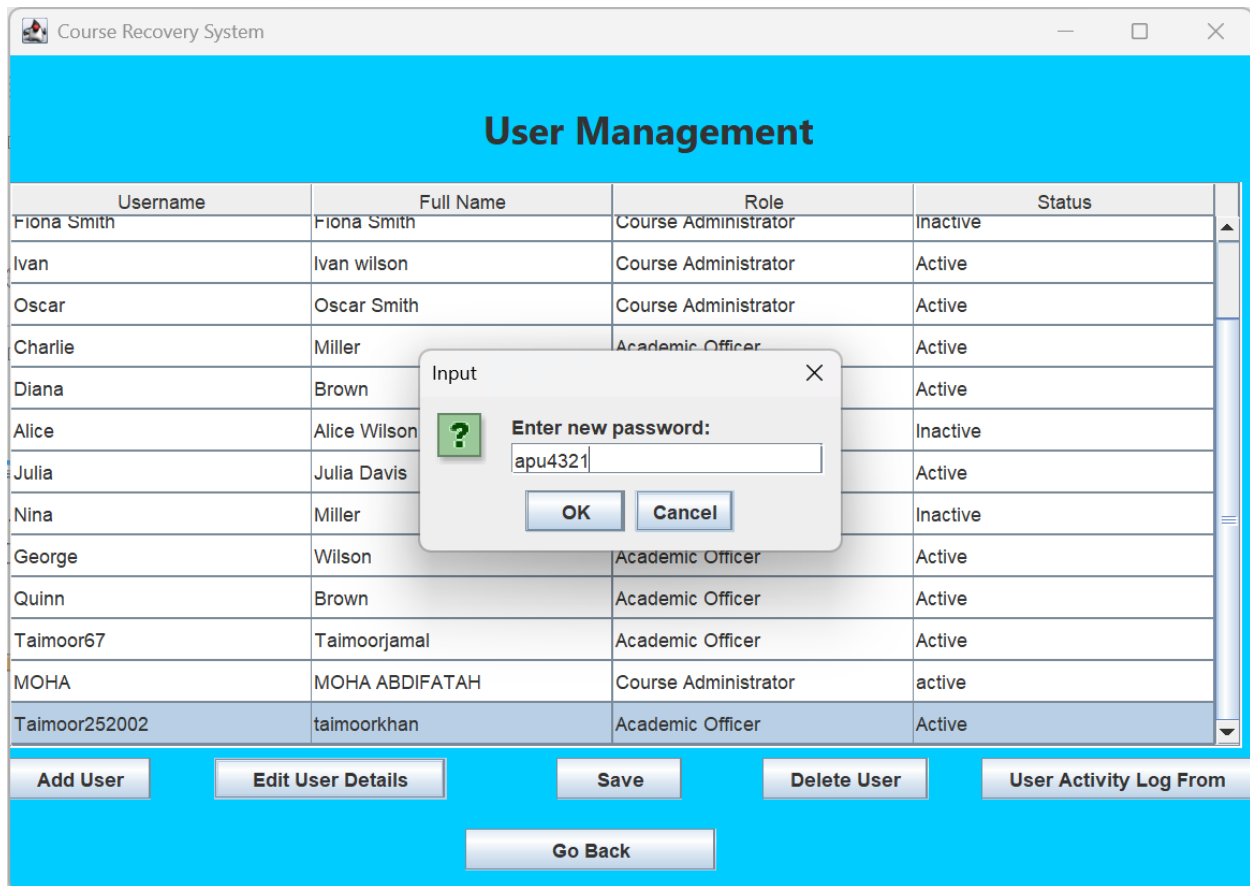
Username	Full Name	Role	Status
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan Wilson	Course Administrator	Active
Oscar	Oscar Smith	Course Administrator	Active
Charlie	Miller	Academic Officer	Active
Diana	Brown	Academic Officer	Active
Alice	Alice Wilson	Course Administrator	Inactive
Julia	Julia Davis	Academic Officer	Active
Nina	Miller		Inactive
George	Wilson		Active
Quinn	Brown		Active
Taimoor67	Taimoorjama		Active
MOHA	MOHA ABDIFATTAH	Course Administrator	active
Taimoor252002	taimoor jAMAL	Course Administrator	Inactive

Message
 ✕


Current password is incorrect.

Error message shown when incorrect old password is entered.

Change Password interface requesting old and new credentials.



The screenshot shows a web application window titled "Course Recovery System". The main content area has a blue header with the text "User Management". Below the header is a table with four columns: Username, Full Name, Role, and Status. The table contains 15 rows of user data. A modal dialog box is open in the center of the screen, titled "Input" with a close button (X). The dialog contains a green question mark icon, the text "Enter new password:", a text input field with the value "apu4321", and two buttons: "OK" and "Cancel". At the bottom of the main interface, there are five buttons: "Add User", "Edit User Details", "Save", "Delete User", and "User Activity Log From". A "Go Back" button is also present at the bottom center.

Username	Full Name	Role	Status
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan Wilson	Course Administrator	Active
Oscar	Oscar Smith	Course Administrator	Active
Charlie	Miller	Academic Officer	Active
Diana	Brown		Active
Alice	Alice Wilson		Inactive
Julia	Julia Davis		Active
Nina	Miller		Inactive
George	Wilson	Academic Officer	Active
Quinn	Brown	Academic Officer	Active
Taimoor67	Taimoorjamal	Academic Officer	Active
MOHA	MOHA ABDIFATAH	Course Administrator	active
Taimoor252002	taimookhan	Academic Officer	Active

Buttons at the bottom: Add User, Edit User Details, Save, Delete User, User Activity Log From, Go Back.

This screen ensures secure password updates by requiring the current password before allowing changes.

Course Recovery System

User Management

Username	Full Name	Role	Status
Fiona Smith	Fiona Smith	Course Administrator	Inactive
Ivan	Ivan Wilson	Course Administrator	Active
Oscar	Oscar Smith	Course Administrator	Active
Charlie	Miller	Academic Officer	Active
Diana	Brown		Active
Alice	Alice Wilson		Inactive
Julia	Julia Davis		Active
Nina	Miller		Inactive
George	Wilson	Academic Officer	Active
Quinn	Brown	Academic Officer	Active
Taimoor67	Taimoorjamal	Academic Officer	Active
MOHA	MOHA ABDIFATAH	Course Administrator	active
Taimoor252002	taimoorkhan	Academic Officer	Active

Message

Password updated successfully.

OK

Add User Edit User Details Save Delete User User Activity Log From

Go Back

Successful password update confirmation

3.1.5 User Activity Log

 Course Recovery System—□×

User Activity Log

Start Date

End Date

0

0

Filter

Title 1	Title 2	Title 3	Title 4

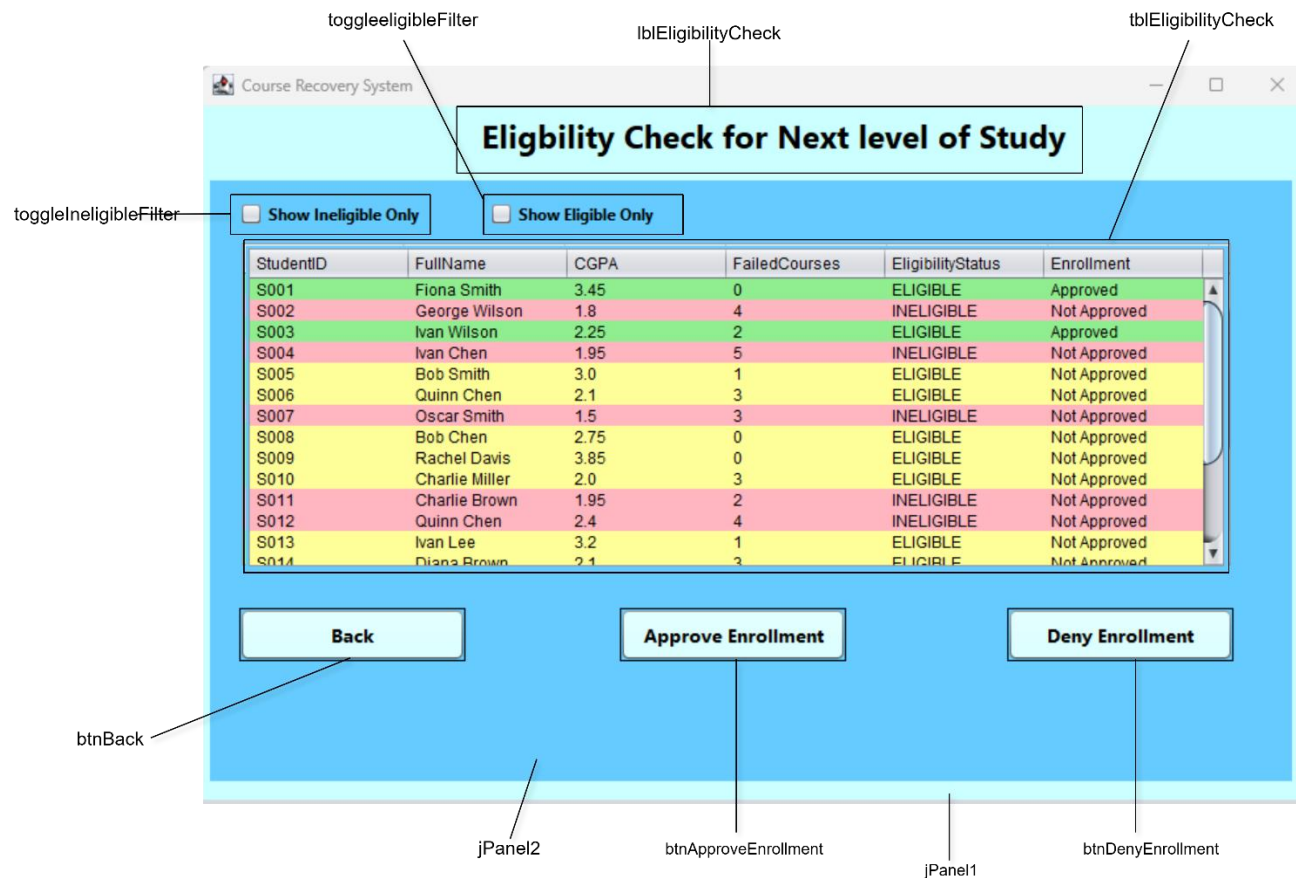
Export Log

Back

User Activity Log displaying recorded system actions.

3.2 Eligibility Check and Enrolment

3.2.1 Main Graphical User Interface



GUI Element	Component Name	Type	Purpose
Main Window	EligibilityCheck	JFrame	Acts as the main window for the Eligibility Check module. It contains all panels, tables, buttons, and labels related to eligibility and enrollment.
Main Container Panel	jPanel1	JPanel	Serves as the container for entire GUI and provides structural organization and background styling.

Content Panel	jPanel2	JPanel	Contains interactive components such as the table and buttons. This panel visually separates functionality from the header.
Title Label	lblEligibilityCheck	JLabel	Displays the title “Eligibility Check for Next Level of Study” to clearly indicate the purpose of the screen.
Student Eligibility Table	tblEligibilityCheck	JTable	Displays records including Student ID, Full Name, CGPA, Failed Courses, Eligibility Status, and Enrollment status.
Approve Enrollment Button	btnApproveEnrollment	JButton	Allows user to approve enrollment for a selected student who meets eligibility criteria. A confirmation dialog is shown before approval.
Deny Enrollment Button	btnDenyEnrollment	JButton	Allows the user to deny or revoke enrollment for a selected student.
Ineligible Filter	toggleIneligibleFilter	JCheckBox	Filters the table to only show students who are ineligible and hides eligible students
Eligible Filter	toggleEligibleFilter	JCheckBox	Filters the table to only show students who are eligible and hides ineligible students

Eligibility Check GUI provides a clear interface for managing student progression decisions. A JTable is used to display academic and enrollment data in a structured format, while buttons allow administrators to approve or deny enrollment. Color-coded rows enhance usability by visually distinguishing students, reducing the need for manual inspection.

 Course Recovery System

—
□
×

Eligibility Check for Next level of Study

☒ Show Ineligible Only
 ☐ Show Eligible Only

StudentID	FullName	CGPA	FailedCourses	EligibilityStatus	Enrollment
S002	George Wilson	1.8	4	INELIGIBLE	Not Approved
S004	Ivan Chen	1.95	5	INELIGIBLE	Not Approved
S007	Oscar Smith	1.5	3	INELIGIBLE	Not Approved
S011	Charlie Brown	1.95	2	INELIGIBLE	Not Approved
S012	Quinn Chen	2.4	4	INELIGIBLE	Not Approved
S015	Alice Wilson	1.7	5	INELIGIBLE	Not Approved
S018	Julia Davis	2.5	4	INELIGIBLE	Not Approved
S020	Rachel Miller	1.6	2	INELIGIBLE	Not Approved

Back

Approve Enrollment

Deny Enrollment

 Course Recovery System

—
□
×

Eligibility Check for Next level of Study

☐ Show Ineligible Only
 ☒ Show Eligible Only

StudentID	FullName	CGPA	FailedCourses	EligibilityStatus	Enrollment
S001	Fiona Smith	3.45	0	ELIGIBLE	Approved
S003	Ivan Wilson	2.25	2	ELIGIBLE	Approved
S005	Bob Smith	3.0	1	ELIGIBLE	Not Approved
S006	Quinn Chen	2.1	3	ELIGIBLE	Not Approved
S008	Bob Chen	2.75	0	ELIGIBLE	Not Approved
S009	Rachel Davis	3.85	0	ELIGIBLE	Not Approved
S010	Charlie Miller	2.0	3	ELIGIBLE	Not Approved
S013	Ivan Lee	3.2	1	ELIGIBLE	Not Approved
S014	Diana Brown	2.1	3	ELIGIBLE	Not Approved
S016	Bob Smith	2.85	0	ELIGIBLE	Not Approved
S017	Alice Johnson	3.05	0	ELIGIBLE	Not Approved
S019	Nina Miller	3.65	1	ELIGIBLE	Not Approved

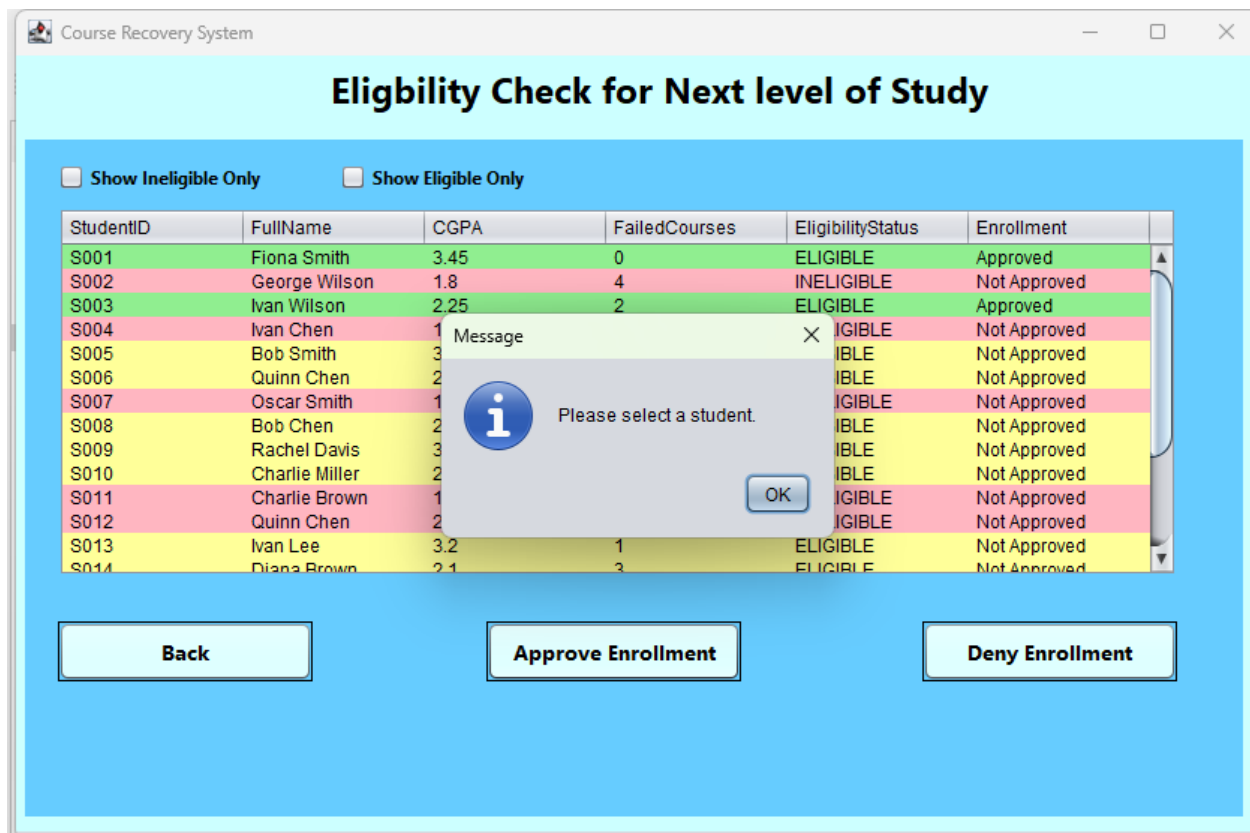
Back

Approve Enrollment

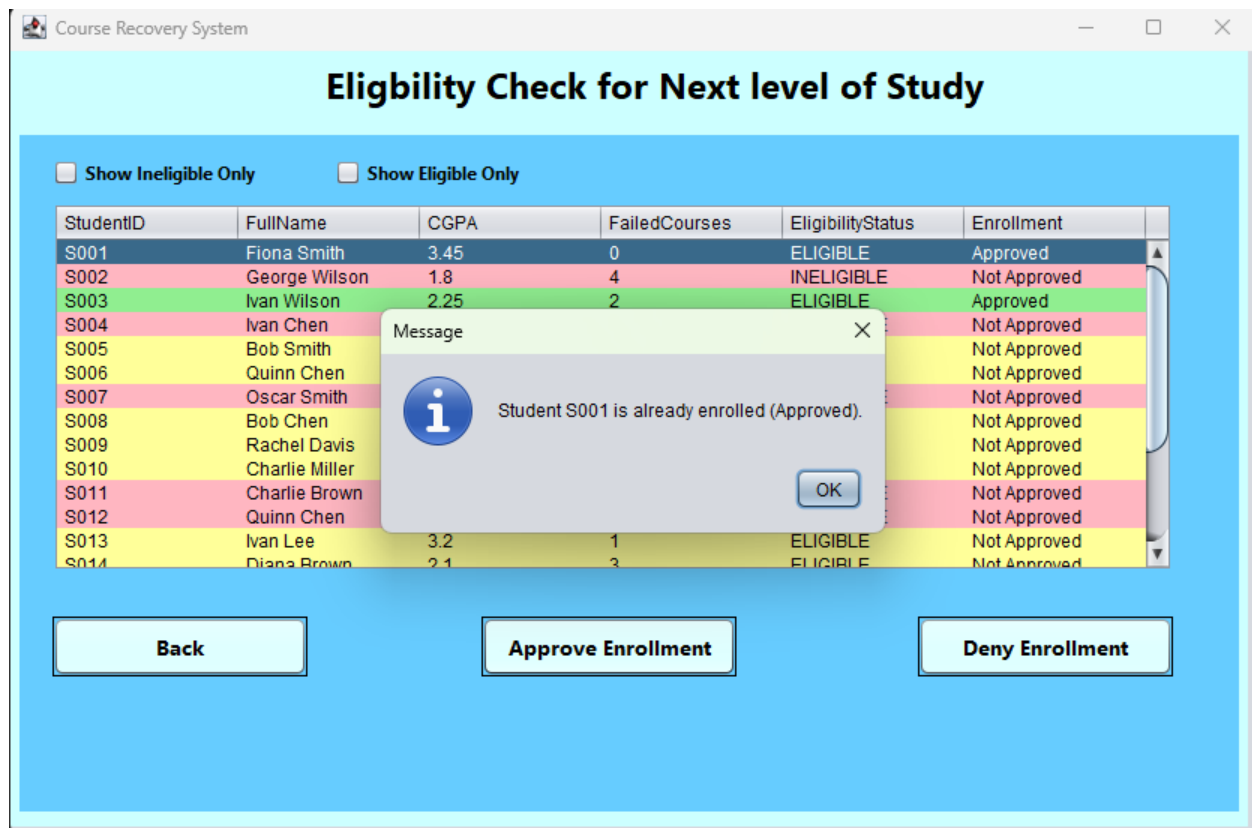
Deny Enrollment

The eligibility filtering feature was implemented using checkbox controls that dynamically adjusted data displayed in the JTable. When a filter is selected, student records are reloaded from the data file and displayed based on their eligibility status, ensuring data integrity while providing a responsive and user-friendly interface enhanced by color indicators.

3.2.3 Approving ineligible and eligible students



When no student is selected but a button is clicked, the system displays a message prompting the user to select a student before continuing preventing null references.



If selected student is already enrolled and 'Approve Enrollment' is clicked, the system shows a message stating the student is already enrolled improving usability.

Course Recovery System

Eligibility Check for Next level of Study

☐ Show Ineligible Only
☐ Show Eligible Only

StudentID	FullName	CGPA	FailedCourses	EligibilityStatus	Enrollment
S001	Fiona Smith	3.45	0	ELIGIBLE	Approved
S002	George Wilson	1.8	4	INELIGIBLE	Not Approved
S003	Ivan Wilson	2.25	2	ELIGIBLE	Approved
S004	Ivan Chen				Not Approved
S005	Bob Smith				Not Approved
S006	Quinn Chen				Not Approved
S007	Oscar Smith				Not Approved
S008	Bob Chen				Not Approved
S009	Rachel Davis				Not Approved
S010	Charlie Miller				Not Approved
S011	Charlie Brown				Not Approved
S012	Quinn Chen				Not Approved
S013	Ivan Lee	3.2	1	ELIGIBLE	Not Approved
S014	Diana Brown	2.1	3	ELIGIBLE	Not Approved

Eligibility Details

Record View:
Student S002 (George Wilson) is INELIGIBLE [CGPA: 1.8, Failed Courses: 4, Enrollment: Not Approved]

Status View:
The student does not currently meet the academic requirements.

OK

Back

Approve Enrollment

Deny Enrollment

Course Recovery System

Eligibility Check for Next level of Study

☐ Show Ineligible Only
☐ Show Eligible Only

StudentID	FullName	CGPA	FailedCourses	EligibilityStatus	Enrollment
S001	Fiona Smith	3.45	0	ELIGIBLE	Approved
S002	George Wilson	1.8	4	INELIGIBLE	Not Approved
S003	Ivan Wilson	2.25	2	ELIGIBLE	Approved
S004	Ivan Chen				Not Approved
S005	Bob Smith				Not Approved
S006	Quinn Chen				Not Approved
S007	Oscar Smith				Not Approved
S008	Bob Chen				Not Approved
S009	Rachel Davis				Not Approved
S010	Charlie Miller				Not Approved
S011	Charlie Brown				Not Approved
S012	Quinn Chen				Not Approved
S013	Ivan Lee	3.2	1	ELIGIBLE	Not Approved
S014	Diana Brown	2.1	3	ELIGIBLE	Not Approved

Message

This student is not eligible and cannot be approved.

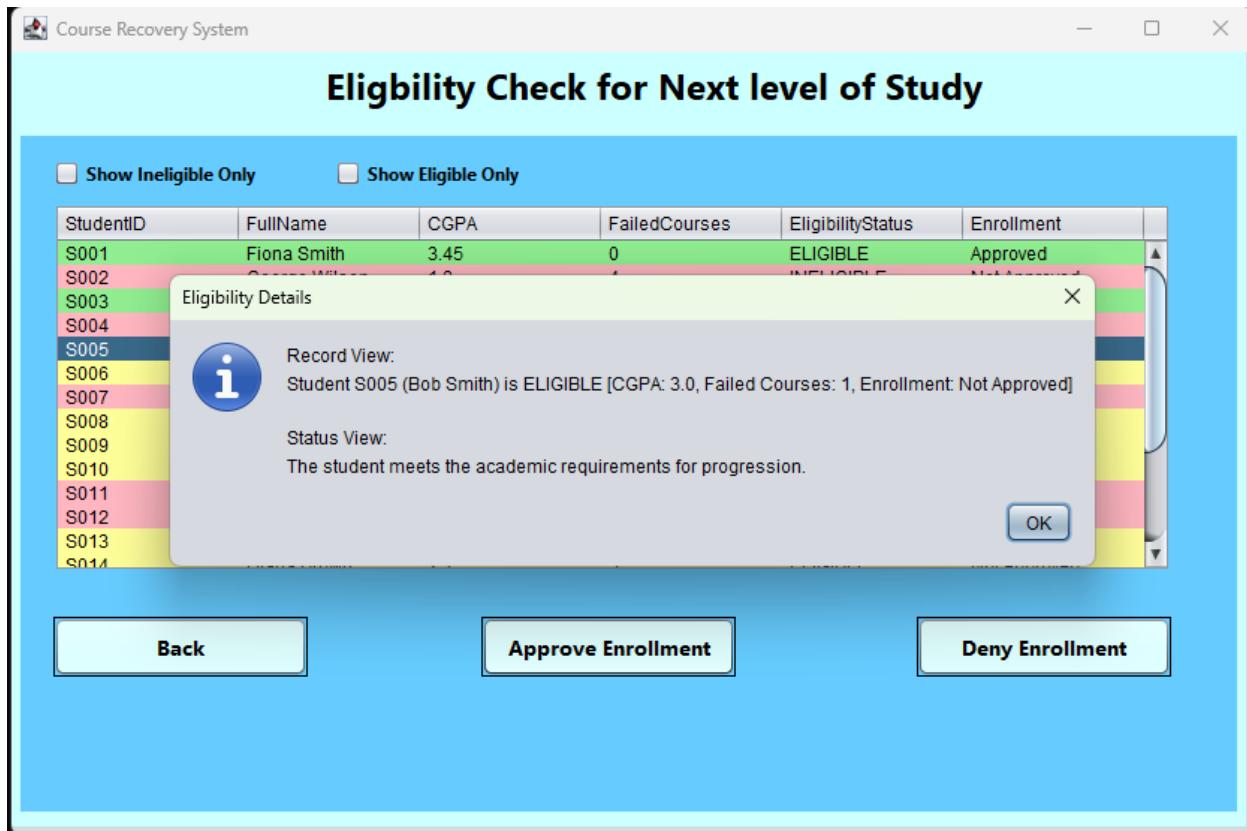
OK

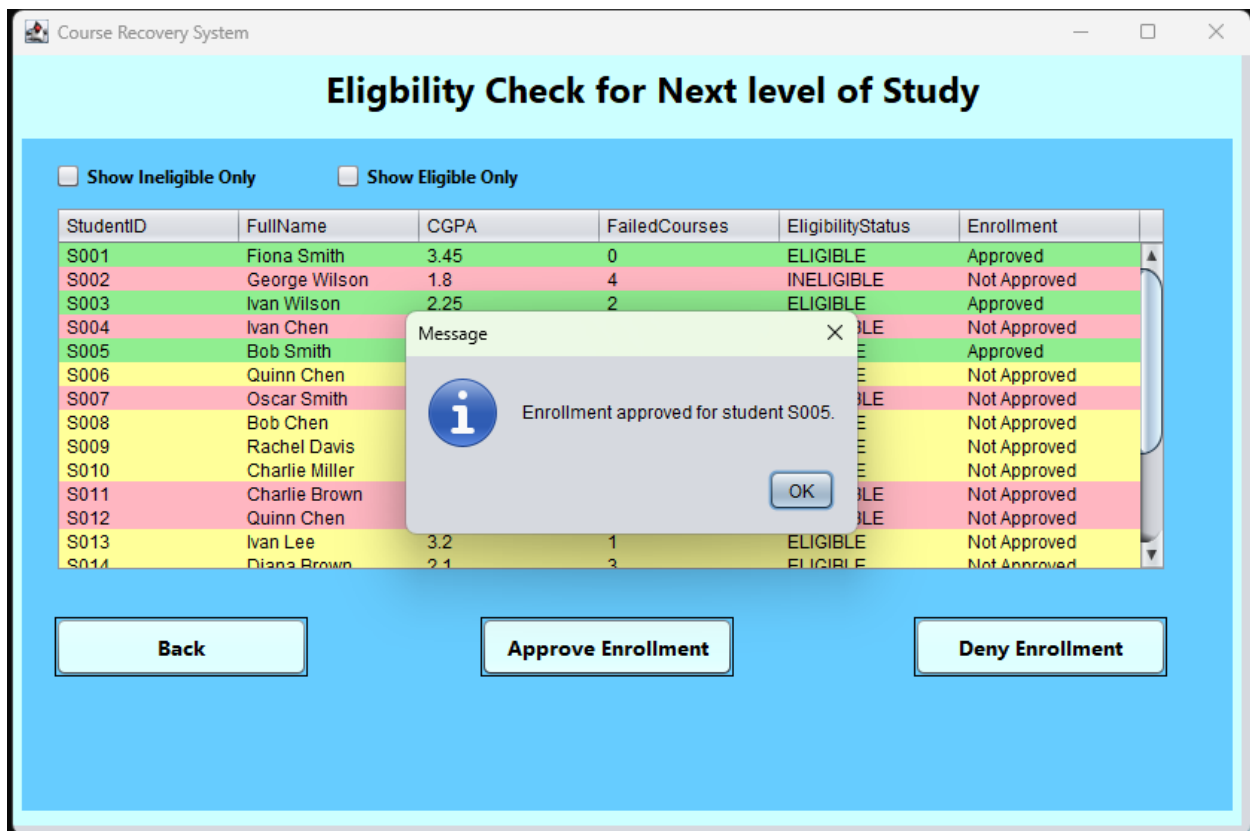
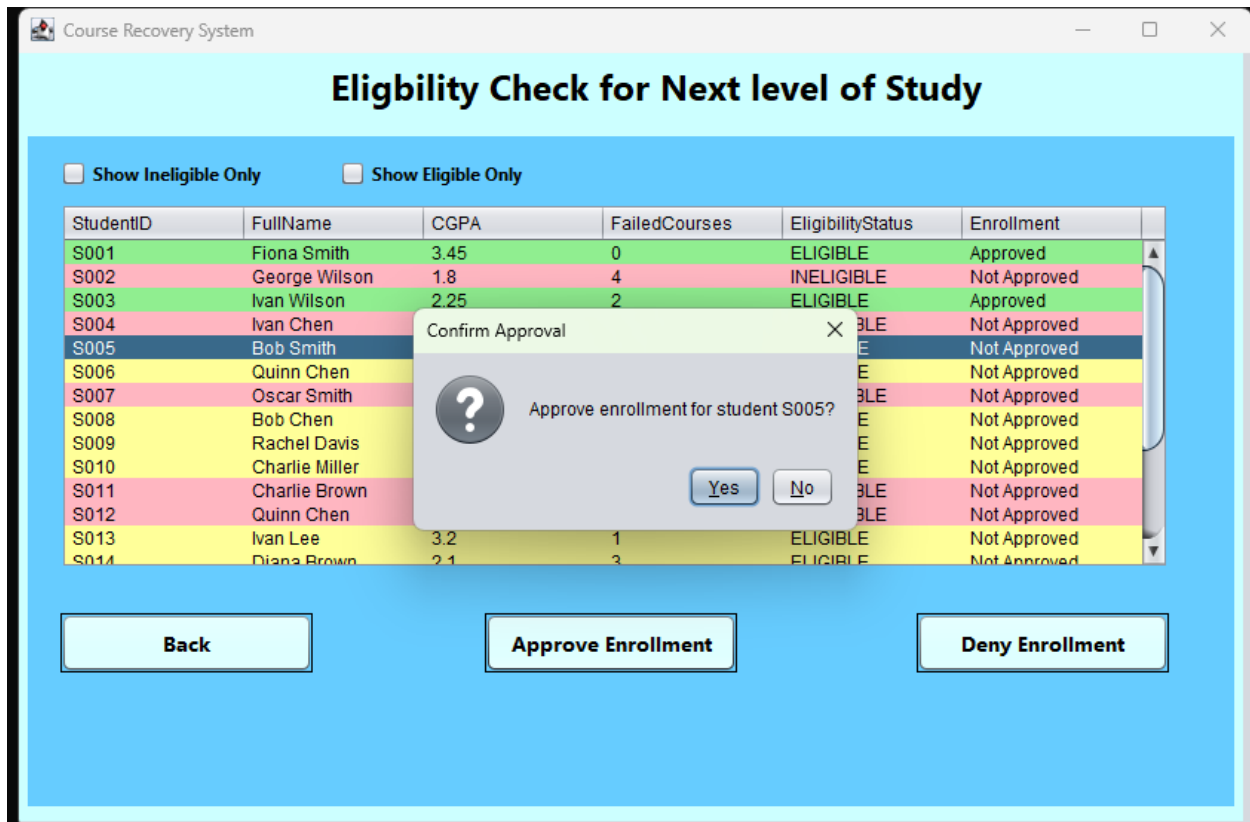
Back

Approve Enrollment

Deny Enrollment

When 'Approve Enrollment' is select the system retrieves all information to check whether the selected student is allowed to be approved, but in this case the student selected is ineligible so a dialog box shows up and shows that this student can't be approved because he is ineligible and therefore nothing happens.

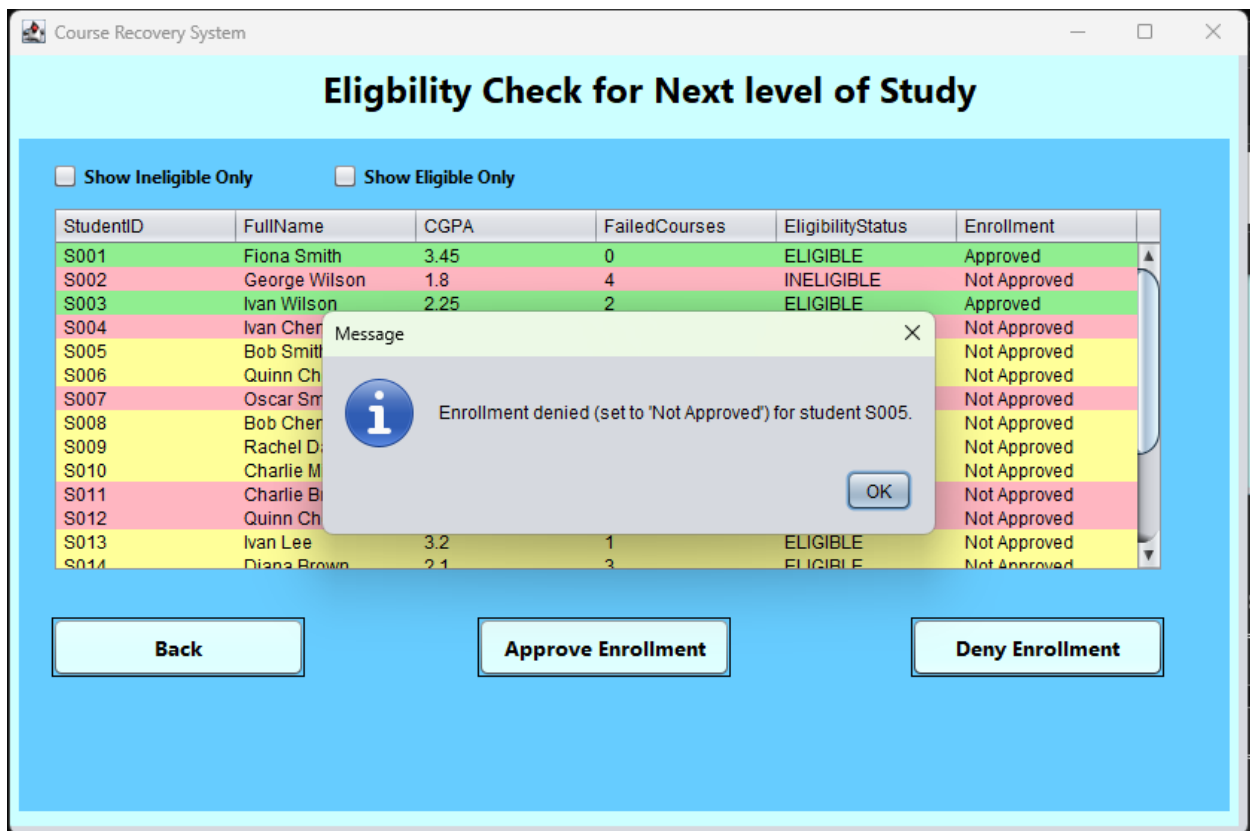




Over here when selected student is eligible and 'Approve Enrollment' is selected a dialog

message pops up and shows showing CGPA, failed courses, eligible status and enrollment and as shown the student is eligible, so when clicked the jtable updates automatically and the enrollment column updates to 'Approved' and this change is also saved in the eligibility_records.txt, but this happens only after confirmation from the user.

3.2.4 Denying Enrolment for Eligible and Ineligible Students



For eligible students who are already approved, when 'Deny Enrollment' is selected a confirmation box asks the user to confirm whether he wants to revoke the enrollment as this student is eligible to prevent clicking by mistake and invalid operations.

Course Recovery System

Eligibility Check for Next level of Study

☐ Show Ineligible Only
☐ Show Eligible Only

StudentID	FullName	CGPA	FailedCourses	EligibilityStatus	Enrollment
S001	Fiona Smith	3.45	0	ELIGIBLE	Approved
S002	George Wilson	1.8	4	INELIGIBLE	Not Approved
S003	Ivan Wilson	2.25	2	ELIGIBLE	Approved
S004	Ivan Che				Not Approved
S005	Bob Smith				Not Approved
S006	Quinn Ch				Not Approved
S007	Oscar S				Not Approved
S008	Bob Che				Not Approved
S009	Rachel D				Not Approved
S010	Charlie N				Not Approved
S011	Charlie E				Not Approved
S012	Quinn Ch				Not Approved
S013	Ivan Lee	3.2	1	ELIGIBLE	Not Approved
S014	Diana Brown	2.1	3	ELIGIBLE	Not Approved

Message

i Enrollment is already set to 'Not Approved' for student S004.

OK

Back

Approve Enrollment

Deny Enrollment

For students who are ineligible or eligible but are already set to 'Not Approved' a dialog box shows that the student selected is already 'Not Approved' preventing duplicate entries.

3.2.5 Back button

Course Recovery System

Eligibility Check for Next level of Study

☐ Show Ineligible Only ☐ Show Eligible Only

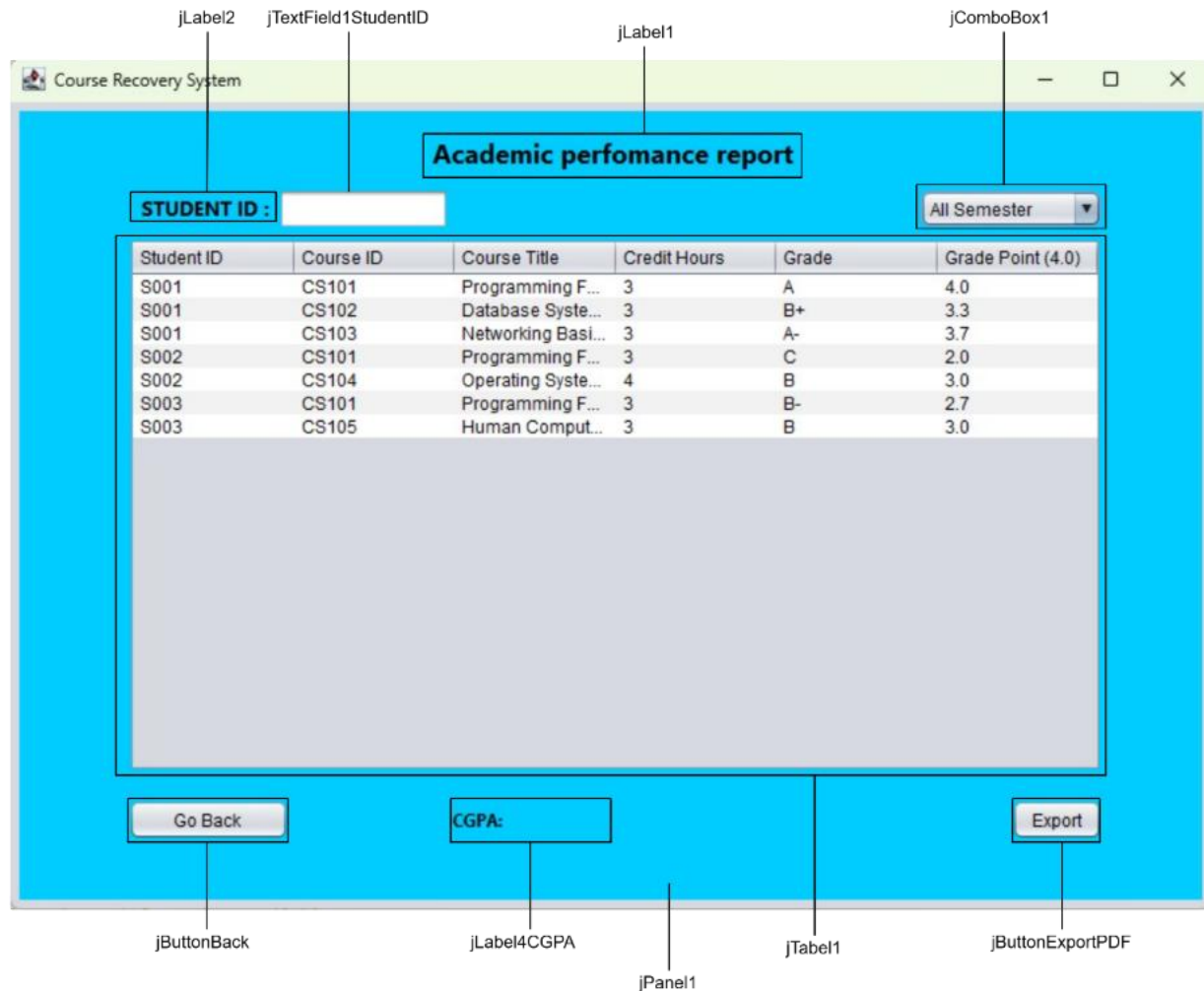
StudentID	FullName	CGPA	FailedCourses	EligibilityStatus	Enrollment
S001	Fiona Smith	3.45	0	ELIGIBLE	Approved
S002	George Wilson	1.8	4	INELIGIBLE	Not Approved
S003	Ivan Wilson	2.25	2	ELIGIBLE	Approved
S004	Ivan Chen	1.95	5	INELIGIBLE	Not Approved
S005	Bob Smith	3.0	1	ELIGIBLE	Not Approved
S006	Quinn Chen	2.1	3	ELIGIBLE	Not Approved
S007	Oscar Smith	1.5	3	INELIGIBLE	Not Approved
S008	Bob Chen	2.75	0	ELIGIBLE	Not Approved
S009	Rachel Davis	3.85	0	ELIGIBLE	Not Approved
S010	Charlie Miller	2.0	3	ELIGIBLE	Not Approved
S011	Charlie Brown	1.95	2	INELIGIBLE	Not Approved
S012	Quinn Chen	2.4	4	INELIGIBLE	Not Approved
S013	Ivan Lee	3.2	1	ELIGIBLE	Not Approved
S014	Diana Brown	2.1	3	ELIGIBLE	Not Approved

Back **Approve Enrollment** **Deny Enrollment**

When the back button is selected, the system changes the frame to the previous one, which is the dashboard for the Academic Officer, making navigation easier and more user friend

3.3 Academic Performance Report

3.3.1 Main GUI Components

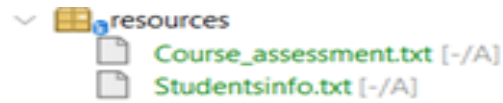


GUI Element	Element Name (Code)	Type	Purpose
Main Window	AcademicperformanceGUI	JFrame	Acts as the main window for academic reporting
Main Panel	jPanel1	JPanel	Root container providing layout structure

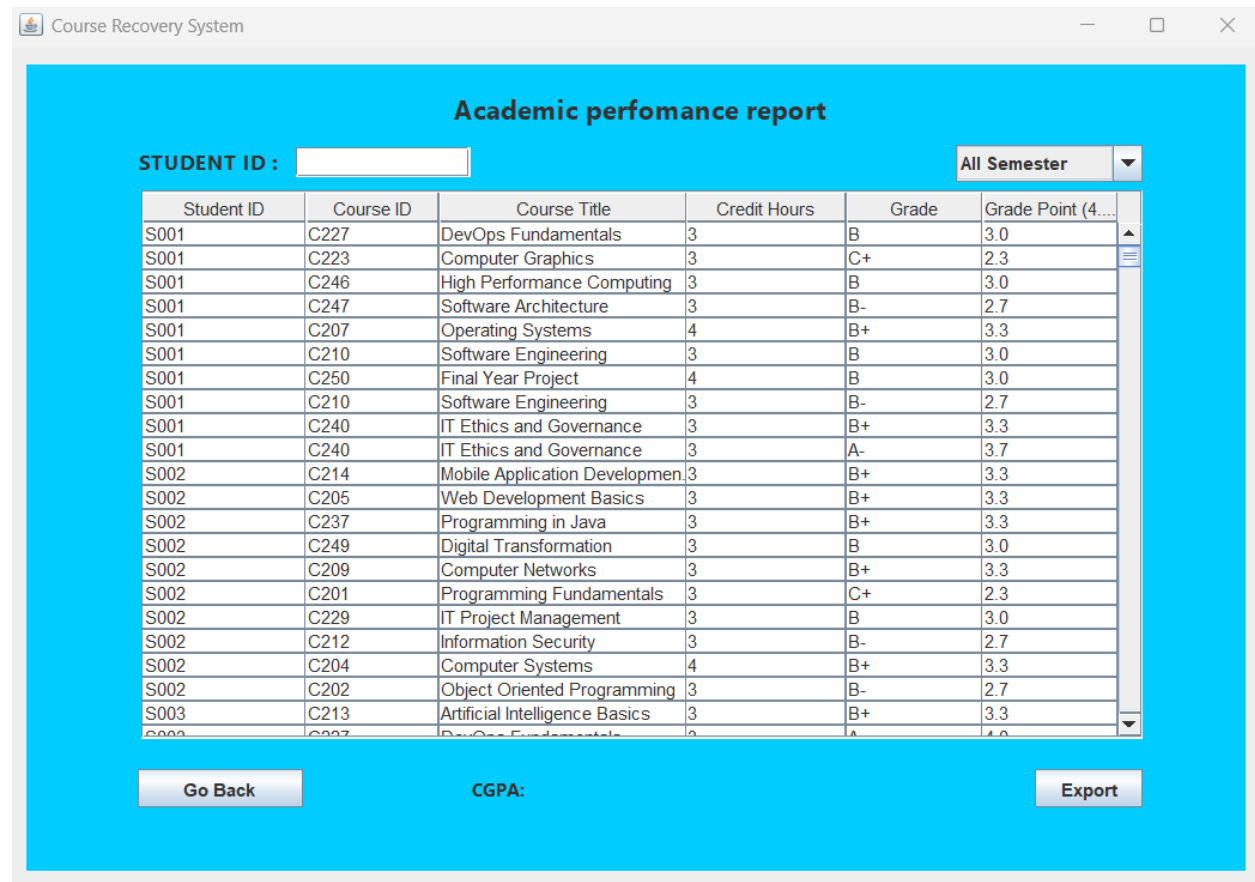
GUI Element	Element Name (Code)	Type	Purpose
Student ID Input	jTextField1StudentID	JTextField	Accepts Student ID and triggers student search
Semester Filter	jComboBox1	JComboBox	Filters academic results by semester
Results Table	jTable1	JTable	Displays academic results
Scroll Pane	jScrollPane1	JScrollPane	Enables scrolling for large datasets
CGPA Display	jLabel4CGPA	JLabel	Displays calculated GPA/CGPA
Export Button	jButtonExportPDF	JButton	Generates and exports PDF report
Back Button	jButtonBack	JButton	Close the reporting window
Title Label	jLabel1	JLabel	Displays screen title

3.3.2 System Flow

1. When deployed, the Academic Performance Reporting module extracts student records of a student of **Studentsinfo.txt** and course assessment records of a course of **Course_assessment.txt**.



2. The system Shows the **Academic Performance Reporting GUI**, which acts as the main interaction interface. Where all the data of the files are loaded



3. The user enters a **Student ID** into the input field.

The screenshot shows a web application window titled "Course Recovery System". Inside, there's a section titled "Academic performance report". Below this title, there's a form with "STUDENT ID : S067" and a dropdown menu set to "All Semester". A table displays the student's performance across various courses. The table has six columns: Student ID, Course ID, Course Title, Credit Hours, Grade, and Grade Point (4.0). Below the table, there's a large empty rectangular box. At the bottom of the interface, there are three elements: a "Go Back" button, a "CGPA: 3.15" label, and an "Export" button.

Student ID	Course ID	Course Title	Credit Hours	Grade	Grade Point (4.0)
S067	C232	Software Testing	3	A	4.0
S067	C247	Software Architecture	3	C+	2.3
S067	C233	Blockchain Fundamentals	3	C+	2.3
S067	C220	Digital Logic Design	4	A	4.0
S067	C233	Blockchain Fundamentals	3	A-	3.7
S067	C213	Artificial Intelligence Basics	3	A	4.0
S067	C222	Enterprise Systems	3	C+	2.3
S067	C220	Digital Logic Design	4	B	3.0
S067	C229	IT Project Management	3	B	3.0
S067	C201	Programming Fundamentals	3	B-	2.7

The system searches for the in-memory student list and identifies the matching student record.

4. The user selects a **semester option** (*All Semester, Semester 1, or Semester 2*) from the dropdown list.

The screenshot shows a web application window titled "Course Recovery System". Inside, there's a section titled "Academic performance report". Below this title, there's a form with a "STUDENT ID" field containing "S067" and a "Semester" dropdown menu currently set to "All Semester". Below the form is a table displaying academic records. The table has six columns: Student ID, Course ID, Course Title, Credit Hours, Grade, and Grade Point (4.0). The table contains 12 rows of data for student S067. Below the table, there's a large empty rectangular area. At the bottom of the interface, there are three buttons: "Go Back", "CGPA: 3.15", and "Export".

Student ID	Course ID	Course Title	Credit Hours	Grade	Grade Point (4.0)
S067	C232	Software Testing	3	A	4.0
S067	C247	Software Architecture	3	C+	2.3
S067	C233	Blockchain Fundamentals	3	C+	2.3
S067	C220	Digital Logic Design	4	A	4.0
S067	C233	Blockchain Fundamentals	3	A-	3.7
S067	C213	Artificial Intelligence Basics	3	A	4.0
S067	C222	Enterprise Systems	3	C+	2.3
S067	C220	Digital Logic Design	4	B	3.0
S067	C229	IT Project Management	3	B	3.0
S067	C201	Programming Fundamentals	3	B-	2.7

5. The system filters the academic records based on the selected semester and displays the results in a **JTable**.

Course Recovery System

Academic performance report

STUDENT ID : S067

Semester 1

Student ID	Course ID	Course Title	Credit Hours	Grade	Grade Point (4.0)
S067	C220	Digital Logic Design	4	A	4.0
S067	C233	Blockchain Fundamentals	3	A-	3.7
S067	C229	IT Project Management	3	B	3.0
S067	C201	Programming Fundamentals	3	B-	2.7

Go Back

CGPA: 3.40

Export

Semester 1

Course Recovery System

Academic performance report

STUDENT ID : Semester 2

Student ID	Course ID	Course Title	Credit Hours	Grade	Grade Point (4.0)
S067	C232	Software Testing	3	A	4.0
S067	C247	Software Architecture	3	C+	2.3
S067	C233	Blockchain Fundamentals	3	C+	2.3
S067	C213	Artificial Intelligence Basics	3	A	4.0
S067	C222	Enterprise Systems	3	C+	2.3
S067	C220	Digital Logic Design	4	B	3.0

Go Back CGPA: 2.98 Export

Semester 2

7. The system calculates the **GPA/CGPA** dynamically and displays the value below the results table. (Down the **jtable**)

8. The academic report is exported by clicking on the Export button initiated by the user.

9. It checks the identification of the student and corresponding academic information before the system creates a PDF file with the **Academic Performance Report** with the help of the **iText** library. The document that is generated is then placed on the desktop of the user, and the name of the file is that of the semester.

Structure of Generated PDF Report

The generated PDF report contains the following sections:

- **Report Title**
“Academic Performance Report” shows at the top.

Student Information Section

Includes:

- Student ID
- Student Name
- Gender
- Year of Study

Course Results Table

Displays:

- Course Code
- Course Title
- Credit Hours
- Grade
- Grade Point (4.0 scale)

CGPA Summary

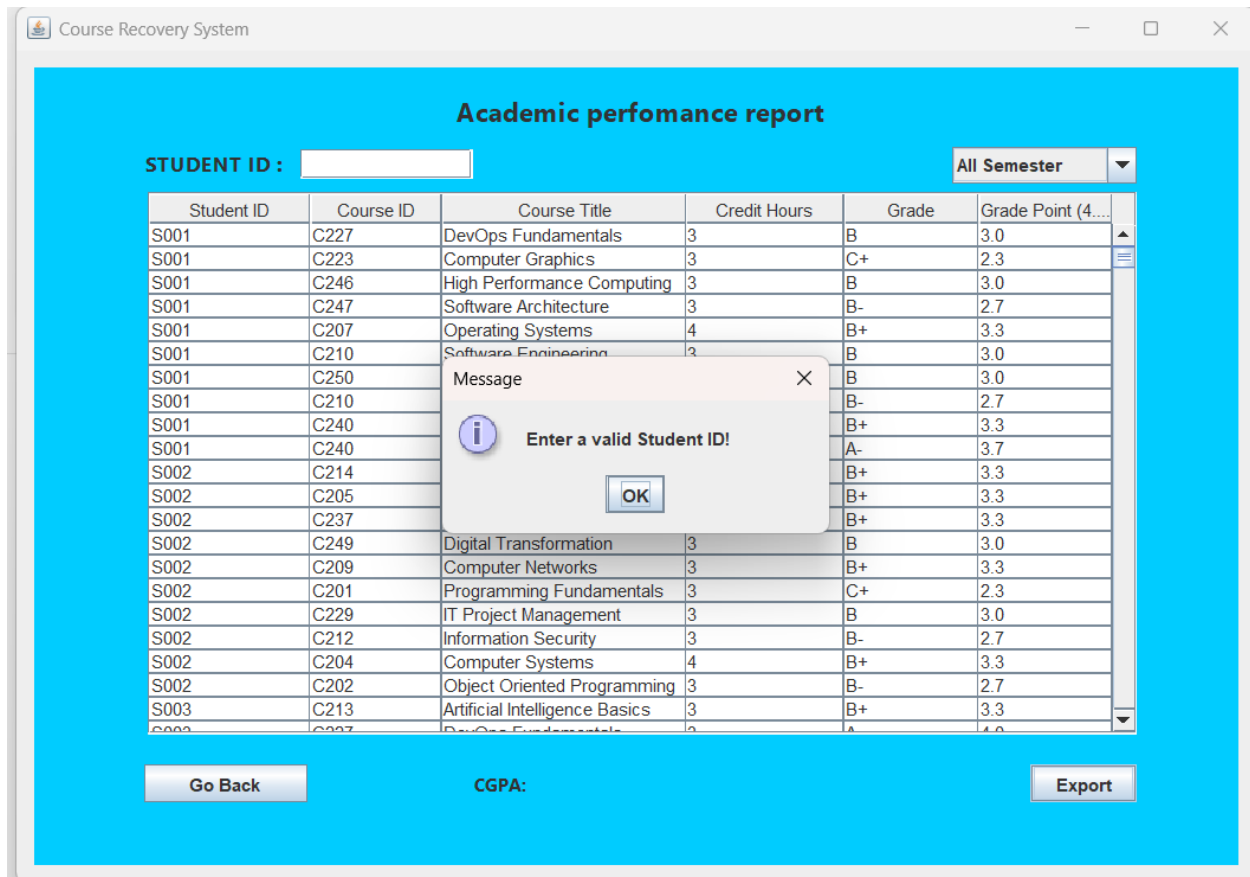
The final CGPA value is clearly highlighted at the bottom of the report.

This structured format ensures clarity, professionalism, and academic validity.

Academic Performance Report				
Student ID:	S067			
Name:	Nathan Goh			
Gender:	Male			
Year:	Year 3			
Course Code	Course Title	Credit Hours	Grade	Grade Point (4.0 Scale)
C232	Software Testing	3	A	4.0
C247	Software Architecture	3	C+	2.3
C233	Blockchain Fundamentals	3	C+	2.3
C220	Digital Logic Design	4	A	4.0
C233	Blockchain Fundamentals	3	A-	3.7
C213	Artificial Intelligence Basics	3	A	4.0
C222	Enterprise Systems	3	C+	2.3
C220	Digital Logic Design	4	B	3.0
C229	IT Project Management	3	B	3.0
C201	Programming Fundamentals	3	B-	2.7
CGPA: 3.15				

Post conditions

1. If the Student ID is empty or invalid during export, the system displays an error message and stops the export process.



2. If no Student ID is entered, the system allows browsing of all academic records without calculating CGPA.

Course Recovery System

Academic performance report

STUDENT ID :
All Semester

Student ID	Course ID	Course Title	Credit Hours	Grade	Grade Point (4...
S001	C227	DevOps Fundamentals	3	B	3.0
S001	C223	Computer Graphics	3	C+	2.3
S001	C246	High Performance Computing	3	B	3.0
S001	C247	Software Architecture	3	B-	2.7
S001	C207	Operating Systems	4	B+	3.3
S001	C210	Software Engineering	3	B	3.0
S001	C250	Final Year Project	4	B	3.0
S001	C210	Software Engineering	3	B-	2.7
S001	C240	IT Ethics and Governance	3	B+	3.3
S001	C240	IT Ethics and Governance	3	A-	3.7
S002	C214	Mobile Application Development	3	B+	3.3
S002	C205	Web Development Basics	3	B+	3.3
S002	C237	Programming in Java	3	B+	3.3
S002	C249	Digital Transformation	3	B	3.0
S002	C209	Computer Networks	3	B+	3.3
S002	C201	Programming Fundamentals	3	C+	2.3
S002	C229	IT Project Management	3	B	3.0
S002	C212	Information Security	3	B-	2.7
S002	C204	Computer Systems	4	B+	3.3
S002	C202	Object Oriented Programming	3	B-	2.7
S003	C213	Artificial Intelligence Basics	3	B+	3.3
S003	C227	DevOps Fundamentals	3	A	4.0

Go Back
CGPA:
Export

3.4 Course Recovery Plan

3.4.1 Filtered Class Test View

The screenshot displays the 'Course Recovery System' window. The main heading is 'COURSE RECOVERY PLAN FOR FAILED STUDENTS'. Below this, the student's name is 'George Wilson' and the major is 'Literature'. A 'Reset Filter' button is located to the right. The 'COMPONENTS FAILED:' section contains a table with four columns: COURSE ID, COURSE NAME, GRADE, and FAILED COMPONENT. To the right of the table are two dropdown menus: one showing 'S002' and another labeled 'Filter by Component'. At the bottom, there are two buttons: 'Back' and 'Send Recommendation Email'.

COURSE ID	COURSE NAME	GRADE	FAILED COMPONENT
LIT100	The Road Not Taken	D	Class Test
LIT101	Intro to Shakespeare	F+	Individual Assignment
LIT203	Modern Poetry	F+	Class Test
LIT204	Pride and Prejudice	F	Class Test
LIT305	American Gothic	F-	Class Test
LIT210	The Great Gatsby	D	Class Test
LIT105	World Mythology	F+	Class Test
LIT302	Postcolonial Texts	F	Class Test
LIT308	Creative Writing	F-	Final Exam

This Course Recovery Plan interface enables administrators to review and manage academic failures for specific students. It displays a detailed table of failed courses, grades, and specific assessment components. The system allows users to filter data by component or student ID and includes functionality to add recommendation entries, facilitating structured academic intervention and support planning.

3.4.2 Component Filtering

The screenshot shows a web application window titled "Course Recovery System". The main heading is "COURSE RECOVERY PLAN FOR FAILED STUDENTS". Below this, the student's name is "George Wilson" and the major is "Literature". A "Reset Filter" button is located to the right of the major field. Under the heading "COMPONENTS FAILED:", there is a table with four columns: "COURSE ID", "COURSE NAME", "GRADE", and "FAILED COMPONENT". The table lists eight courses with their respective grades and failed components. To the right of the table, there is a dropdown menu currently showing "S002" and another dropdown menu labeled "Filter by Component". At the bottom of the interface, there are two buttons: "Back" and "Send Recommendation Email".

COURSE ID	COURSE NAME	GRADE	FAILED COMPONENT
LIT100	The Road Not Taken	D	Class Test
LIT101	Intro to Shakespeare	F+	Individual Assignment
LIT203	Modern Poetry	F+	Class Test
LIT204	Pride and Prejudice	F	Class Test
LIT305	American Gothic	F-	Class Test
LIT210	The Great Gatsby	D	Class Test
LIT105	World Mythology	F+	Class Test
LIT302	Postcolonial Texts	F	Class Test
LIT308	Creative Writing	F-	Final Exam

The Course Recovery Plan interface provides a filtering mechanism that allows administrators to sort the student's failure record by specific assessment types. By expanding the **"Filter by Component"** dropdown menu, users can select categories such as **"Individual Assignment," "Group Assignment," "Class Test,"** or **"Final Exam."** This action refines the data displayed in the "Components Failed" table, enabling a targeted analysis of the specific areas where the student is struggling. A **"Reset Filter"** button is also available to quickly restore the full list of data.

3.4.3 Filtered Class Test View

The screenshot shows a web application window titled "Course Recovery System". The main heading is "COURSE RECOVERY PLAN FOR FAILED STUDENTS". Below this, the student information is displayed: "STUDENT NAME: George Wilson" and "MAJOR: Literature". A "Reset Filter" button is located to the right of the major information. Under the heading "COMPONENTS FAILED:", there is a table with four columns: "COURSE ID", "COURSE NAME", "GRADE", and "FAILED COMPONENT". The table lists seven courses, all of which have "Class Test" as the failed component. To the right of the table, there are two dropdown menus: the first is set to "S002" and the second is set to "Class Test". At the bottom of the interface, there are two buttons: "Back" and "Send Recommendation Email".

COURSE ID	COURSE NAME	GRADE	FAILED COMPONENT
LIT100	The Road Not Taken	D	Class Test
LIT203	Modern Poetry	F+	Class Test
LIT204	Pride and Prejudice	F	Class Test
LIT305	American Gothic	F-	Class Test
LIT210	The Great Gatsby	D	Class Test
LIT105	World Mythology	F+	Class Test
LIT302	Postcolonial Texts	F	Class Test

This view of the **Course Recovery Plan** demonstrates the interface results after applying the "Class Test" filter for student S002 (George Wilson). The system updates the Components Failed by student to isolate specific academic struggles, displaying the courses where the student failed the Class Test component. This focused display eliminates unrelated assessment types, allowing administrators to address consistent difficulties in test-taking scenarios.

3.4.4 Student Status Validation

The screenshot displays the 'Course Recovery System' window. The main heading is 'COURSE RECOVERY PLAN FOR FAILED STUDENTS'. Below this, the student's name is 'Bob Smith' and the major is 'Statistics'. A section titled 'COMPONENTS FAILED:' contains a table with columns: COURSE ID, COURSE NAME, GRADE, and FAILED COMPONENT. The table is currently empty. To the right of the table, there is a 'Reset Filter' button, a dropdown menu showing 'S005', and a 'Filter by Component' dropdown. A 'Student Status' pop-up dialog is centered over the table, displaying an information icon and the text: 'Bob Smith with Student ID S005 has passed all courses.' with an 'OK' button. At the bottom of the interface, there are two buttons: 'Back' and 'Send Recommendation Email'.

COURSE ID	COURSE NAME	GRADE	FAILED COMPONENT
-----------	-------------	-------	------------------

The **Course Recovery Plan** module includes an automated status check that notifies administrators when a selected student has successfully completed all academic requirements. As demonstrated in the interface, selecting a student with no failures triggers a pop-up dialog that explicitly confirms, "Bob Smith with Student ID S005 has passed all courses". This immediate validation prevents the creation of unnecessary recovery plans for students who do not require intervention.

3.4.5 Mandatory Selection Validation

The screenshot displays the 'COURSE RECOVERY PLAN FOR FAILED STUDENTS' interface. At the top, the student's name is 'George Wilson' and the major is 'Literature'. Below this is a table titled 'COMPONENTS FAILED:'. The table has four columns: 'COURSE ID', 'COURSE NAME', 'GRADE', and 'FAILED COMPONENT'. The table lists several courses, including LIT100, LIT101, LIT203, LIT204, LIT305, LIT210, LIT105, LIT302, and LIT308. A 'Reset Filter' button is located to the right of the table. Below the table are two buttons: 'Back' and 'Send Recommendation Email'. A 'Selection Required' pop-up window is overlaid on the table, displaying a yellow warning icon and the message 'Please select a course'. The pop-up window has an 'OK' button.

COURSE ID	COURSE NAME	GRADE	FAILED COMPONENT
LIT100	The Road Not Taken	D	Class Test
LIT101	Intro to Shakespeare	F+	Individual Assignment
LIT203	Modern Poetry	F+	Class Test
LIT204	Pride and Prejudice	F	Class Test
LIT305	American Gothic	F-	Class Test
LIT210	The Great Gatsby	D	Class Test
LIT105	World Mythology	F+	Class Test
LIT302	Postcolonial Texts	F	Class Test
LIT308	Creative Writing	F-	Final Exam

The **Course Recovery Plan** implements a safeguard to ensure that remedial actions are correctly linked to specific records. If an administrator attempts to proceed—such as clicking to add a recommendation—without first highlighting a row in the "Components Failed" table, the system blocks the action. A "Selection Required" pop-up window appears with a yellow warning icon and the message **"Please select a course,"** forcing the user to specify which failed component requires intervention before continuing.

3.4.6 Recommendation Through Email

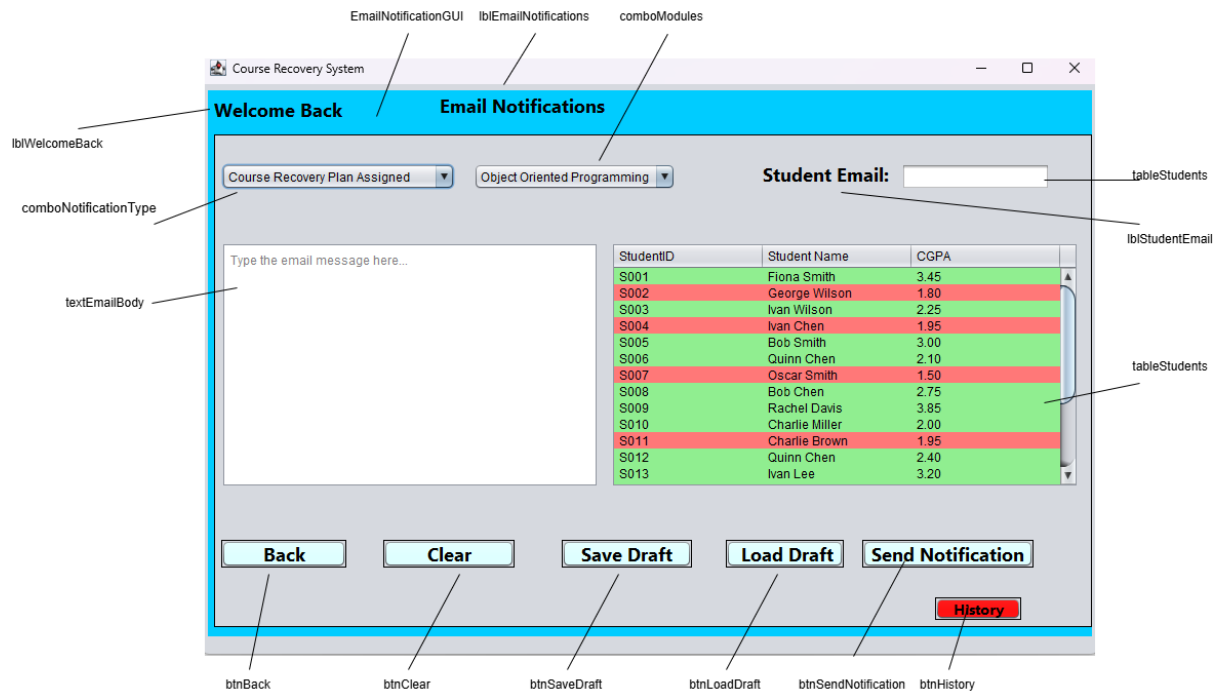
The screenshot shows a web application window titled "Course Recovery System". The main heading is "COURSE RECOVERY PLAN FOR FAILED STUDENTS". Below this, the student's name "Fiona Smith" and major "Computer Science" are displayed. A "Reset Filter" button is located to the right. Under the heading "COMPONENTS FAILED:", there is a table with four columns: COURSE ID, COURSE NAME, GRADE, and FAILED COMPONENT. The table lists four courses: CS201 (Database Systems, D, Individual Assignment), CS202 (Ethical Hacking, F, Group Assignment), CS203 (Python Programming, F+, Individual Assignment), and CS204 (Data Structures, D, Class Test). To the right of the table is a dropdown menu showing "S001" and a "Filter by Component" button. At the bottom, there are two buttons: "Back" and "Send Recommendation Email".

COURSE ID	COURSE NAME	GRADE	FAILED COMPONENT
CS201	Database Systems	D	Individual Assignment
CS202	Ethical Hacking	F	Group Assignment
CS203	Python Programming	F+	Individual Assignment
CS204	Data Structures	D	Class Test

Upon selecting a specific course record and clicking the "**Send Recommendation Email**" button in the Course Recovery Plan, the system transitions to the Email Notification Service interface. This module generates a structured preview of the correspondence, automatically populating the "Course Recovery Assigned" template with the specific module details (e.g., Database Systems) and the student's information.

3.5 Email Notification Service

3.5.1 Main Interface



GUI Element	Variable Name	Type	Purpose
Main Window	EmailNotificationGUI	JFrame	Acts as the main window of the Email Notifications module, containing all panels, controls, and functionality.
Welcome Label	lblWelcomeBack	JLabel	Displays a welcome message to confirm the user has entered the Email Notifications screen.
Title Label	lblEmailNotifications	JLabel	Clearly indicates that the current screen is the Email Notifications module.
Notification Type Selector	comboNotificationType	JComboBox	Allows the user to choose the type of notification, which determines the email template generated.

Module Selector	comboModules	JComboBox	Enables selection of the relevant academic module to contextualize the notification content.
Student Email Label	lblStudentEmail	JLabel	Identifies the text field used for entering the student's email address.
Student Email Field	txtStudentEmail	JTextField	Accepts the student's email address and validates it before sending the notification.
Email Message Area	textEmailBody	JTextArea	Displays the generated email message and supports draft saving and loading.
Student Records Table	tableStudents	JTable	Displays student details (Student ID, Name, CGPA) and allows selection for email personalization.
Back Button	btnBack	JButton	Navigates the user back to the previous screen.
Clear Button	btnClear	JButton	Clears the email message area and shows a message if there is nothing to clear.
Save Draft Button	btnSaveDraft	JButton	Saves the current email message as a draft file for future use.
Load Draft Button	btnLoadDraft	JButton	Opens a dialog to load, delete, or manage saved email drafts.
Send Notification Button	btnSendNotification	JButton	Sends the validated email notification and records the action in the email history log.
History Button	btnHistory	JButton	Opens the email history window showing all sent notifications stored in a text file.

Course Recovery System

Welcome Back

Email Notifications

Milestone Reminder

Object Oriented Programming

Student Email:

Subject: Milestone Reminder

Dear George Wilson (S002),

This is a reminder for your upcoming milestone in Object Oriented Programming.

Please ensure you are following the recovery plan timeline.

Regards,

Course Administrator

StudentID	Student Name	CGPA
S001	Fiona Smith	3.45
S002	George Wilson	1.80
S003	Ivan Wilson	2.25
S004	Ivan Chen	1.95
S005	Bob Smith	3.00
S006	Quinn Chen	2.10
S007	Oscar Smith	1.50
S008	Bob Chen	2.75
S009	Rachel Davis	3.85
S010	Charlie Miller	2.00
S011	Charlie Brown	1.95
S012	Quinn Chen	2.40
S013	Ivan Lee	3.20

Back

Clear

Save Draft

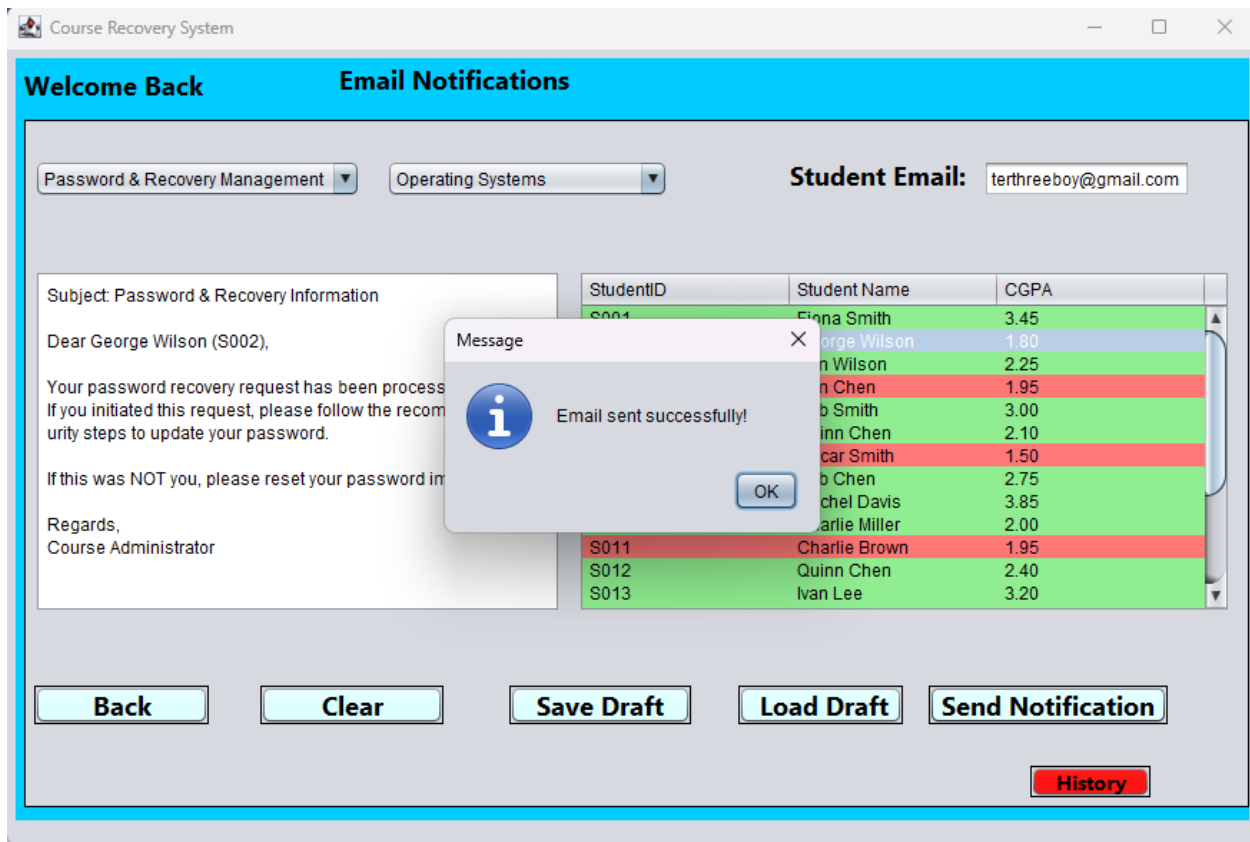
Load Draft

Send Notification

History

Email notification interface enables course administrators to generate and manage structured email notifications for students. It allows selection of notification type and module, automatically generates standardized messages, and links them to student academic data displayed with CGPA-based visual indicators. The system supports secure email sending, draft management, and maintains a history log for accountability and audit purposes.

3.5.2 Email Notification Execution



In this scenario, the administrator selects the notification category and relevant academic module, then chooses a specific student from the table, which auto-personalizes the email content. The student email address is manually entered and validated before sending. Upon successful delivery, the system confirms the action via a popup message “Email sent Successfully”.

Course Notification Inbox x



javasultan81@gmail.com

to me ▼

Subject: Password & Recovery Information

Dear George Wilson (S002),

Your password recovery request has been processed.

If you initiated this request, please follow the recommended security steps to update your password.

If this was NOT you, please reset your password immediately.

Regards,
Course Administrator

The student gets an email confirms that the notification system successfully sent a real email to the student's Gmail inbox. The email content is dynamically generated based on the selected notification type and student details, including name and ID. This demonstrates full integration between the application and Gmail SMTP, proving that notifications are not only simulated but actually delivered to the intended recipient in real time.

- ✓ Dependencies
 - > jakarta.activation-2.0.1.jar
 - > jakarta.mail-2.0.1.jar

jakarta.activation-2.0.1.jar (Jakarta Activation)

This library provides the *data handling layer* used when an email contains different content types (e.g., plain text, attachments, or MIME-based content). Even if your current emails are mostly text, the mail framework still relies on Activation internally to represent and process message content safely and consistently. In your module, it supports the backend capability for composing email bodies in a standard email format that Gmail accepts.

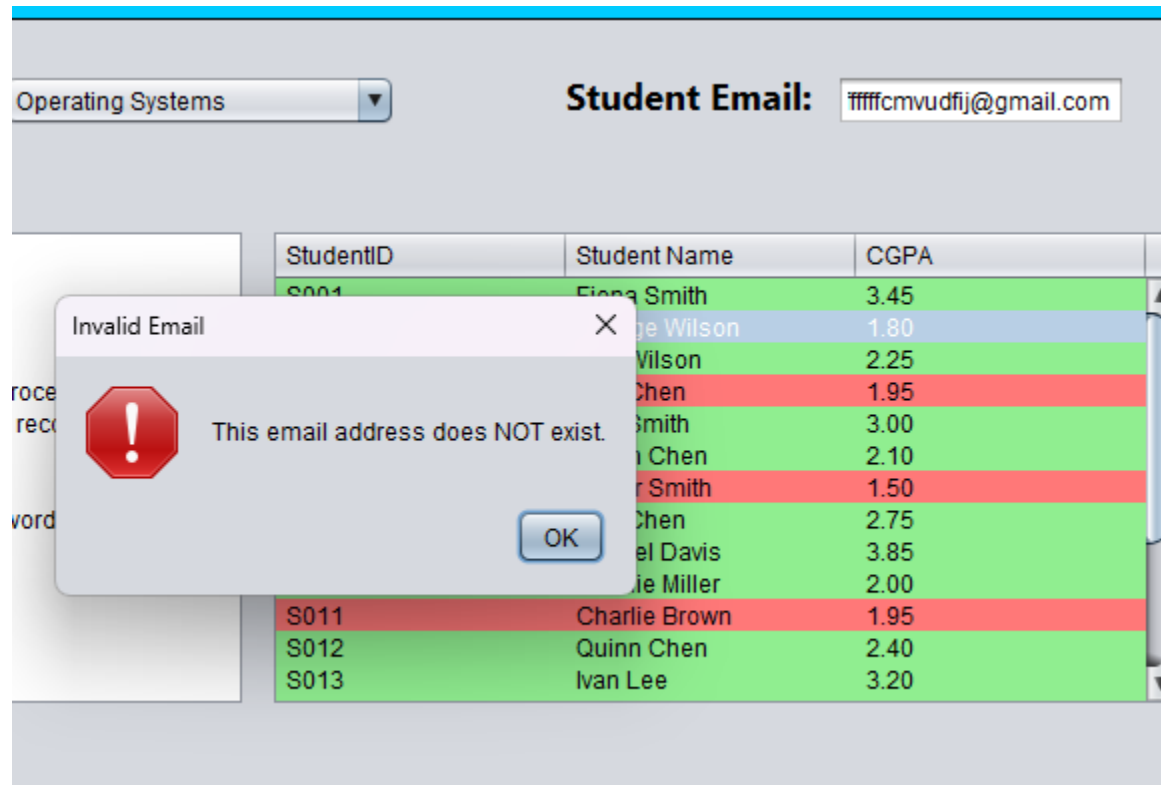
jakarta.mail-2.0.1.jar (Jakarta Mail)

This is the core email-sending library that gives you the classes used directly in your code, such as Session, Message, MimeMessage, Transport, and InternetAddress. In your EmailSender.java, this dependency is what enables the program to: Authenticate to Gmail SMTP (Session.getInstance(...) + Authenticator) ,Construct an email object (MimeMessage) with subject/body and recipient ,Send the email through Gmail (Transport.send(message))

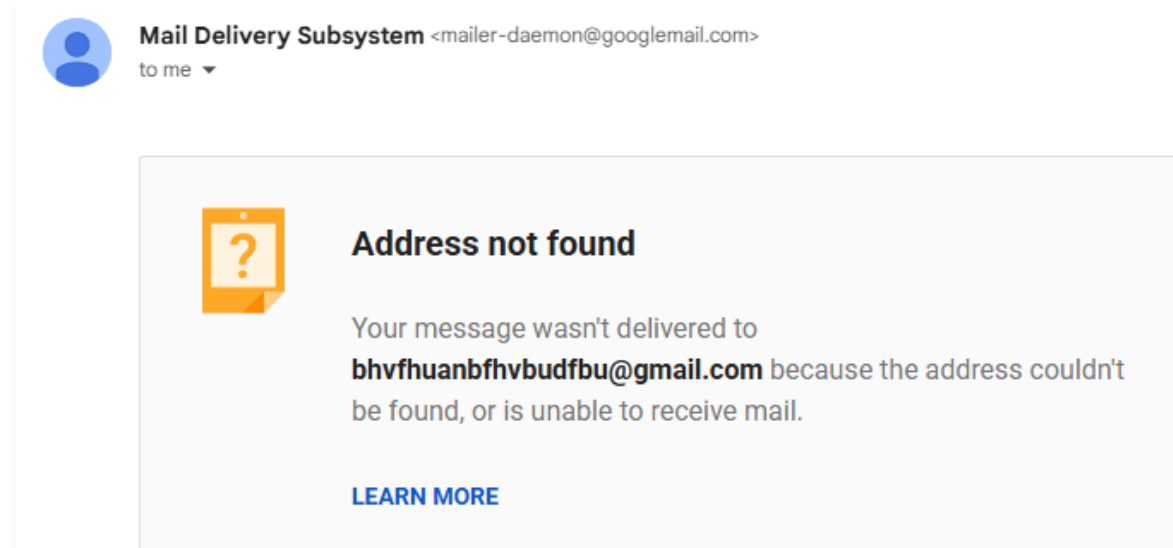
How this connects to your part (Email Notification feature)

These dependencies directly support the Email Notification feature by enabling the system to move beyond a simple GUI simulation into a fully functional communication module. The OOP layer of the system uses the Notification abstract class and its subclasses, created through NotificationFactory, to generate the correct email content using polymorphism via the generateMessage() method. Once the message is generated, the jakarta.mail library handles the actual SMTP communication with Gmail, allowing the system to authenticate, construct, and send real emails to students' Gmail inboxes. The jakarta.activation library supports this process by managing and formatting the email content in a standard MIME-compatible way required by the mail API. Together, these dependencies ensure a complete workflow: selecting a student, generating a notification template, validating the email address, sending the email through Gmail SMTP, and logging the sent action—demonstrating a fully implemented, real-world email notification system.

3.5.3 Invalid Email Handling

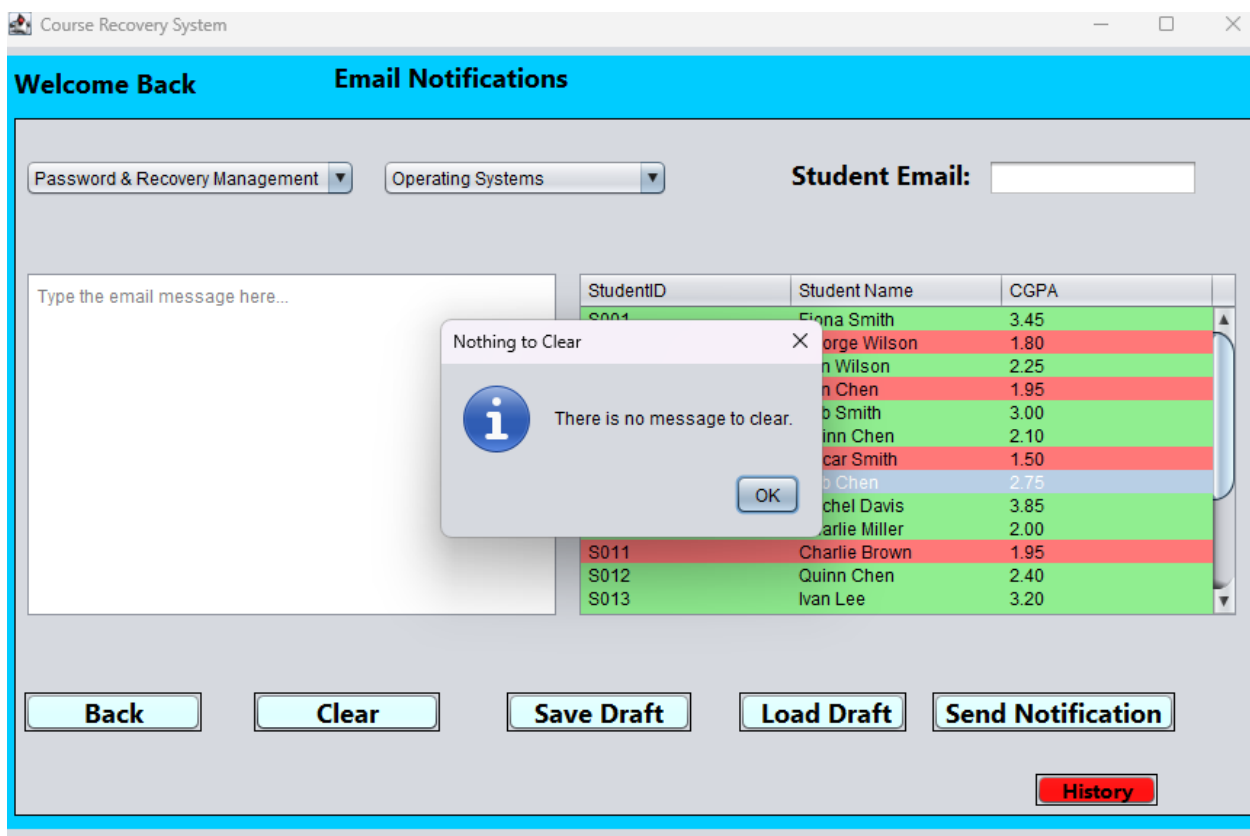


When a non-existent or invalid email address is entered, the system performs validation during the send process and immediately blocks the operation. A clear error popup is displayed to inform the administrator that the email address does not exist.



Automatic response from Gmail's Mail Delivery Subsystem indicating that the entered email address does not exist or cannot receive emails. Gmail attempted to deliver the notification but failed because the recipient address was invalid. This confirms that the system correctly sends emails, and delivery failures are handled externally by Gmail when an address is not found.

3.5.4 Clear Button Behavior



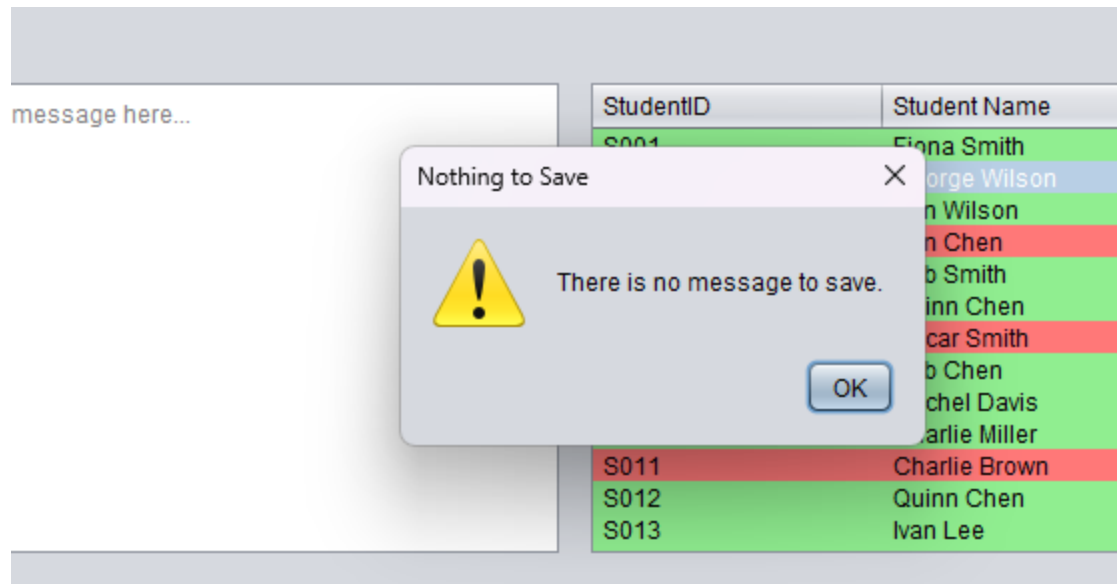
When the Clear button is pressed, the system resets the email text area to its default state by clearing the message content. If the Clear button is pressed again while no message is present, an informational popup is displayed stating that there is nothing to clear.

3.5.5 Save Draft Functionality

The screenshot displays the 'Email Notifications' window of the 'Course Recovery System'. The window has a blue header with 'Welcome Back' and 'Email Notifications'. Below the header, there are two dropdown menus: 'Password & Recovery Management' and 'Operating Systems'. To the right is a 'Student Email:' input field. The main content area is divided into two sections. On the left, an email draft is shown with the subject 'Subject: Password & Recovery Information' and the body text: 'Dear Ivan Wilson (S003), Your password recovery request has been initiated. If you initiated this request, please follow the following steps to update your password. If this was NOT you, please reset your password. Regards, Course Administrator'. On the right, a table lists students with their IDs, names, and CGPAs. A 'Draft Saved' popup is centered over the table, displaying an information icon, the text 'Draft saved as: drafts/S002_Operating_Systems_20251213_113214.draft', and an 'OK' button. At the bottom of the window are five buttons: 'Back', 'Clear', 'Save Draft', 'Load Draft', and 'Send Notification'. A 'History' button is located at the bottom right.

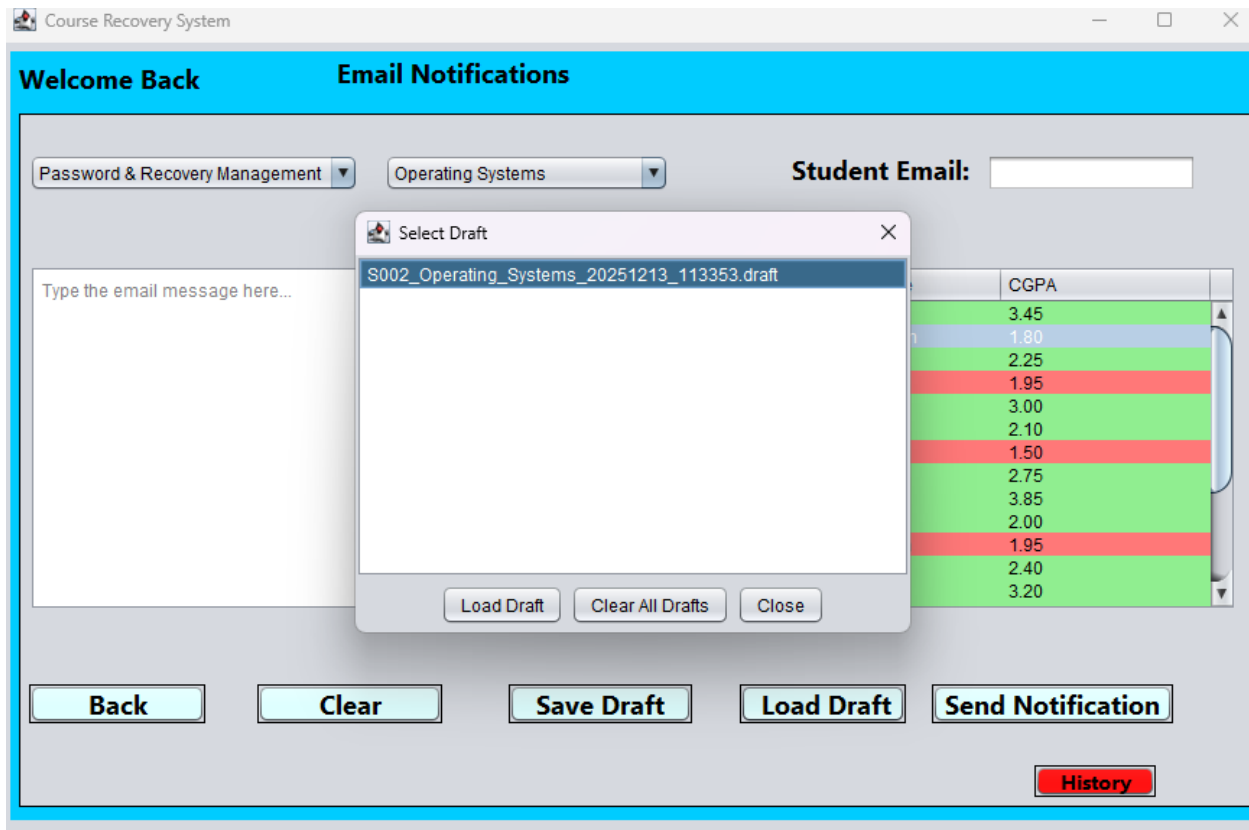
StudentID	Student Name	CGPA
S001	Elena Smith	3.45
S002	John Doe	1.80
S003	Jane Smith	2.25
S004	Mike Johnson	1.95
S005	Sarah Lee	3.00
S006	David Kim	2.10
S007	Emily White	1.50
S008	Chris Brown	2.75
S009	Alex Green	3.85
S010	Mia Black	2.00
S011	Charlie Brown	1.95
S012	Quinn Chen	2.40
S013	Ivan Lee	3.20

The system stores the current email message as a draft file, automatically naming it using the selected student ID, module, date, and a timestamp. A confirmation popup is then displayed showing the exact file path where the draft has been saved. This allows drafts to be safely preserved and retrieved later without sending the email.

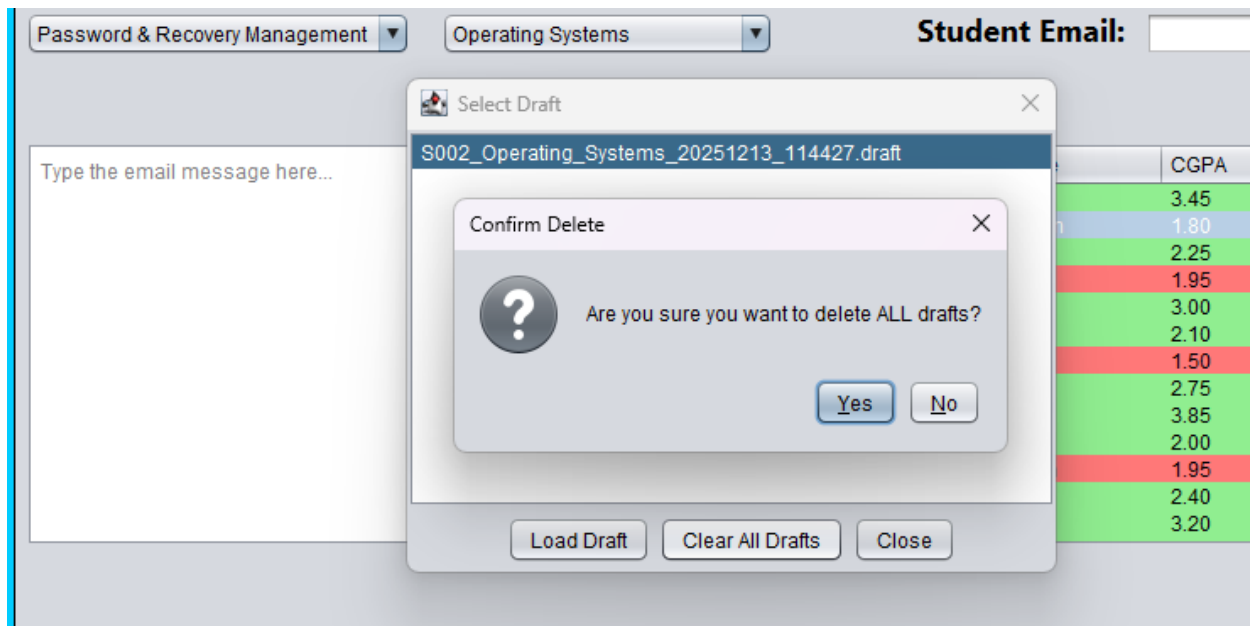


If the Save Draft button is pressed while the message text area is empty (or still showing the placeholder text), the system prevents the action and displays a warning popup stating that there is no message to save. This validation ensures only meaningful email content is stored as draft.

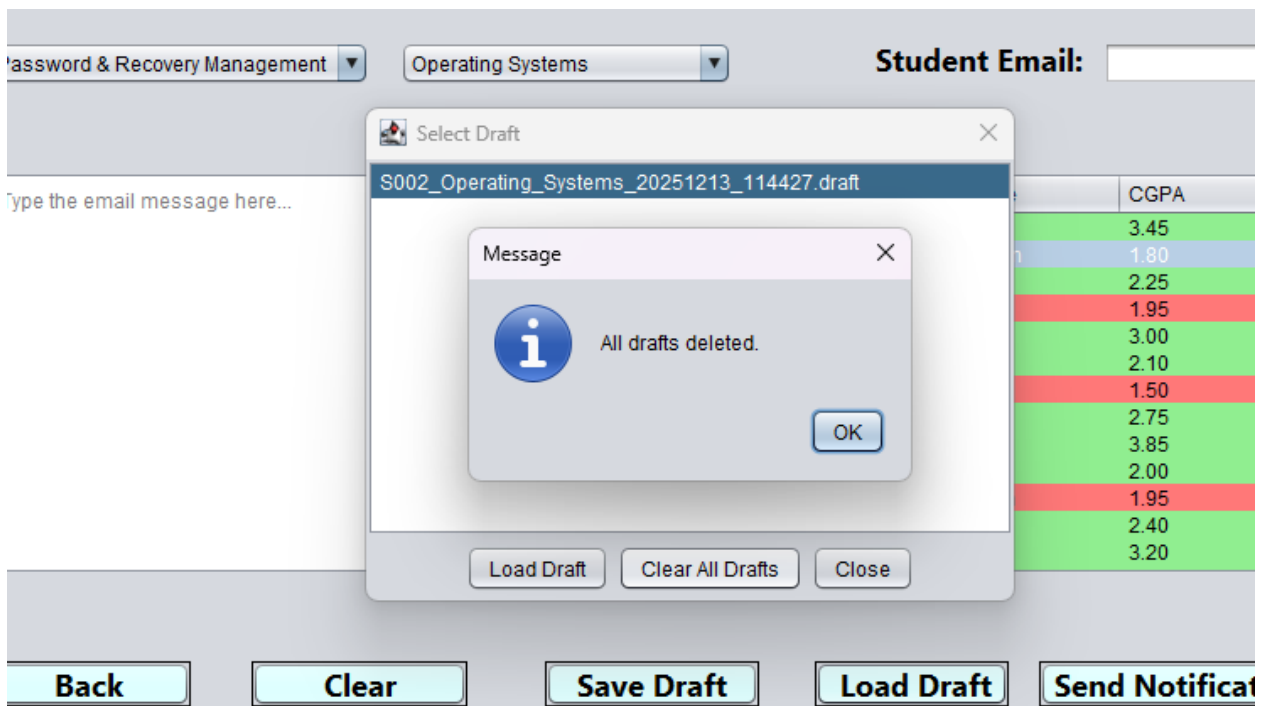
3.5.6 Load Draft Functionality



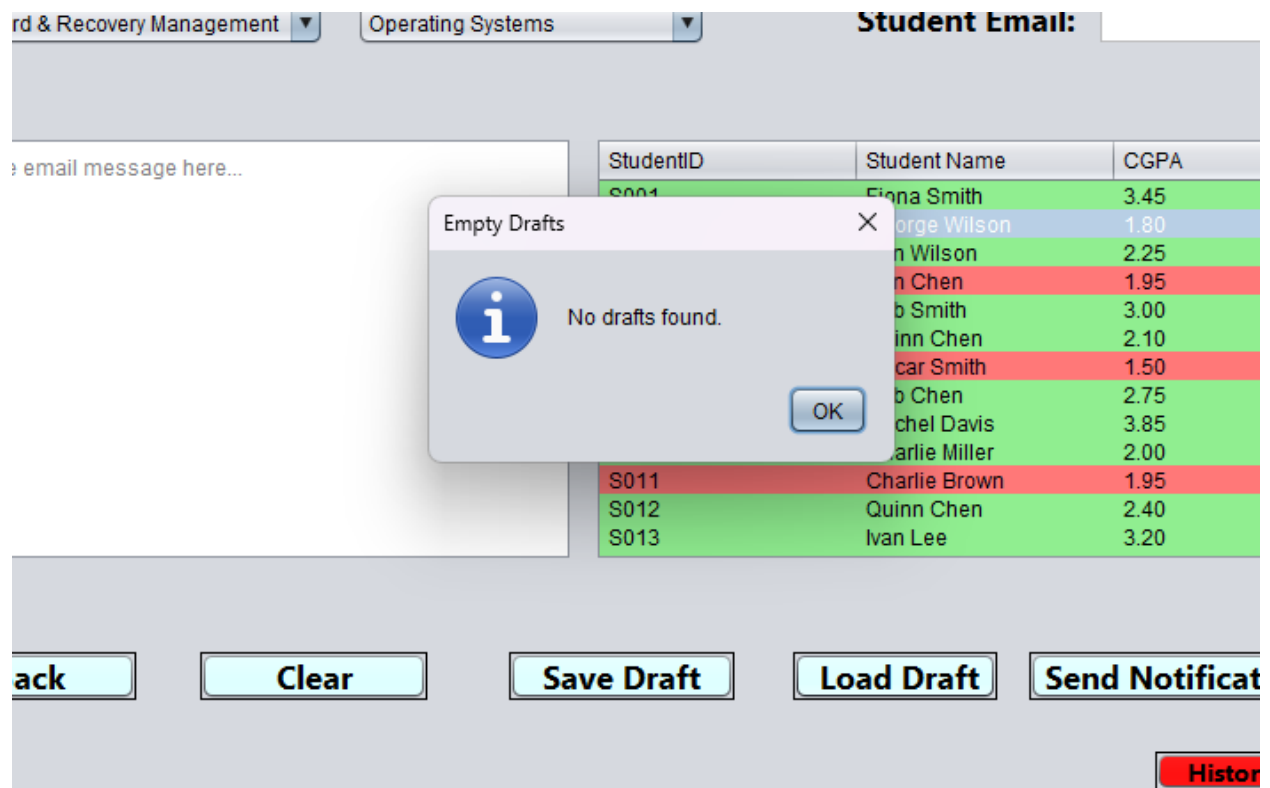
When the Load Draft button is clicked, the system opens a draft selection window displaying all previously saved drafts. The user can choose a draft to load it back into the email text area, delete all drafts to clear stored drafts, or close the window without making any changes. This feature allows efficient reuse and management of saved email drafts.



A confirmation dialog is displayed when the user selects Clear All Drafts. It prevents accidental data loss by requiring explicit user confirmation before permanently deleting all saved drafts.



A pop-up message confirms that all saved draft files have been successfully removed from the system. It provides clear feedback that the delete operation was completed as requested.



A message appears when the user attempts to load drafts, but none exist. It informs the user that there are no saved drafts, ensuring clear system feedback and preventing confusion.

3.5.7 Send Notification button

Course Recovery System

Welcome Back Email Notifications

Password & Recovery Management Operating Systems Student Email:

Subject: Password & Recovery Information

Dear George Wilson (S002),

Your password recovery request has been received. If you initiated this request, please follow the security steps to update your password.

If this was NOT you, please reset your password.

Regards,
Course Administrator

StudentID	Student Name	CGPA
S001	Fiona Smith	3.45
S002	George Wilson	1.80
S003	John Doe	2.25
S004	Jane Doe	1.95
S005	Mike Smith	3.00
S006	Emily White	2.10
S007	David Brown	1.50
S008	Sarah Green	2.75
S009	Robert Black	3.85
S010	Lisa White	2.00
S011	Charlie Brown	1.95
S012	Quinn Chen	2.40
S013	Ivan Lee	3.20

Back Clear Save Draft Load Draft Send Notification

History

Password & Recovery Management Operating Systems Student Email: rthreeboy@hotmail.com

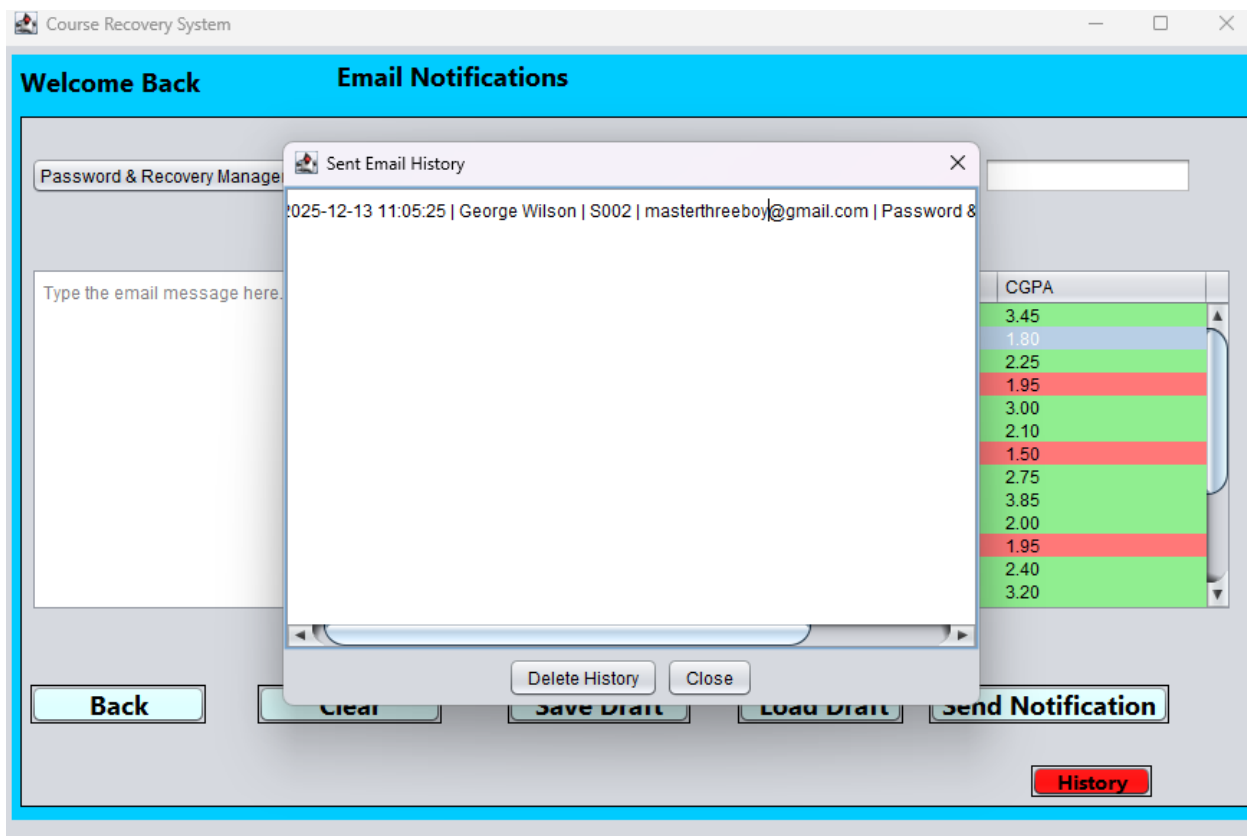
Subject: Password & Recovery Information

George Wilson (S002),

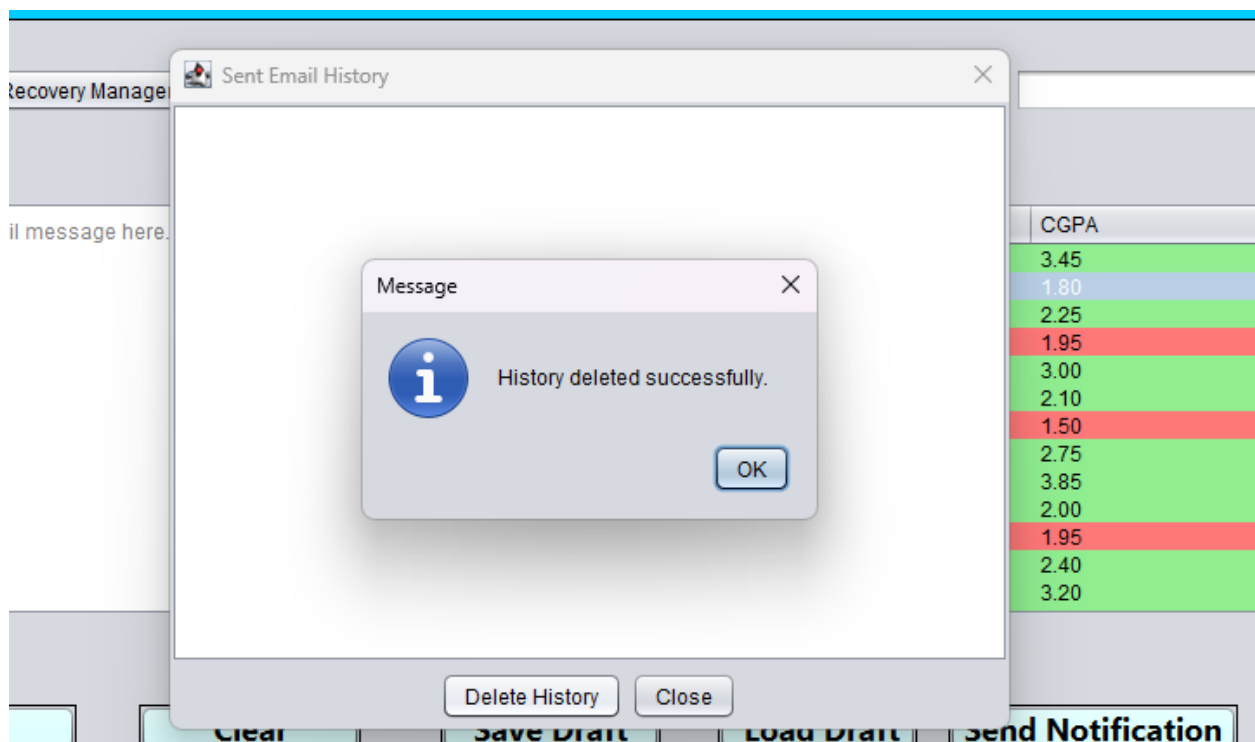
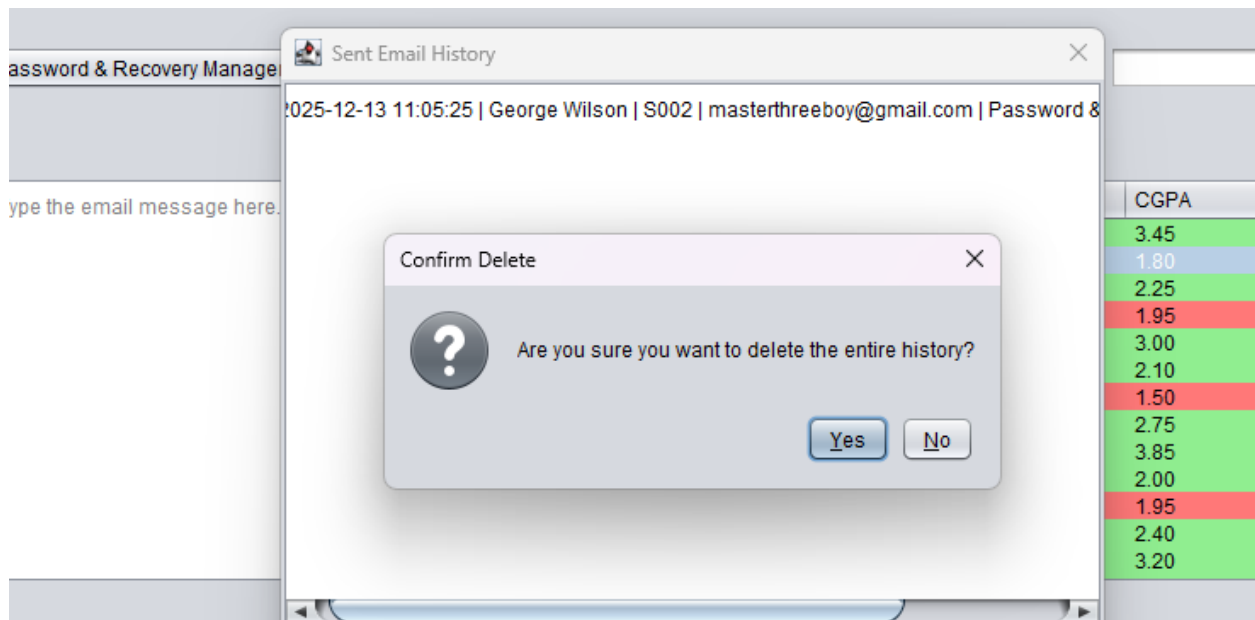
StudentID	Student Name	CGPA
S001	Fiona Smith	3.45
S002	George Wilson	1.80

The system first validates the selected student, email content, and recipient address. If there is nothing in the textbox of the email or the entered email does not end with @gmail.com (e.g., Hotmail or Yahoo), an error popup is displayed, and the email is not sent. This prevents invalid or unsupported email addresses from being processed. Once all validations pass, the notification is sent successfully, and the user receives confirmation feedback.

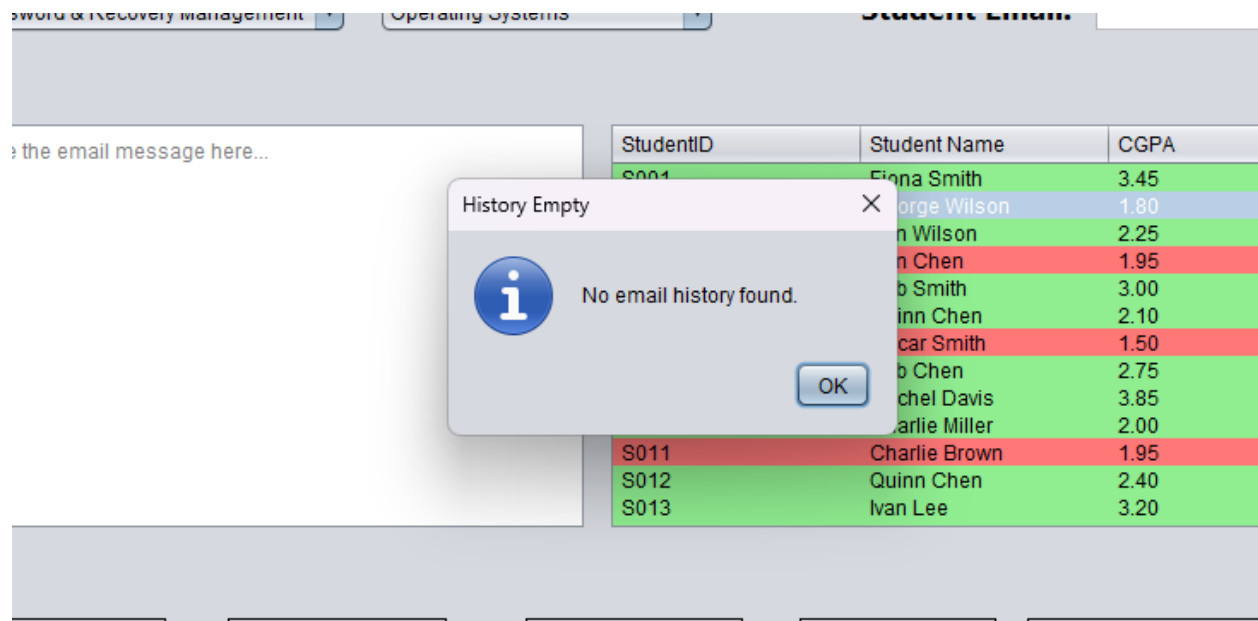
3.5.8 Sent Email History & Audit Log



The History button displays a log of all successfully sent emails in a separate window. Each entry is automatically recorded in a text file, capturing the date and time, student name and ID, recipient email, and notification type. This provides clear traceability and accountability for all notifications sent through the system.



This screen shows the email history management process. When the **Delete History** button is clicked, the system prompts a confirmation dialog to prevent accidental data loss. Upon confirmation, all stored email history records are permanently removed from the text file. A success message is then displayed to clearly inform the user that the history has been deleted successfully.



This message is displayed when there's no email records are available. It informs the user that the email history file is empty or has not yet been created.

CHAPTER 4: OBJECT-ORIENTED PROGRAMMING CONCEPTS

4.1 User Management

4.1.1 Abstraction

```
// ===== LOAD USERS =====
public void loadUsers(DefaultTableModel model) {
    try {
        model.setRowCount(0);
        passwords.clear();

        if (!Files.exists(USERS_FILE)) return;

        List<String> lines = Files.readAllLines(USERS_FILE);
        for (String line : lines) {
            if (line.trim().isEmpty()) continue;

            String[] p = line.split("\\|", -1);
            String username = p[0].trim();
            String fullname = p[1].trim();
            String role      = p[2].trim();
            String status    = p[3].trim();
            String password = p.length >= 5 ? p[4].trim() : ""

            model.addRow(new Object[]{username, fullname, role,
            passwords.put(username, password);
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```

    }

    // ===== SAVE USERS =====
    public void saveUsers(DefaultTableModel model) {
        try (PrintWriter pw = new PrintWriter(USERS_FILE.toFile()))
            for (int i = 0; i < model.getRowCount(); i++) {
                String u = model.getValueAt(i, 0).toString();
                String f = model.getValueAt(i, 1).toString();
                String r = model.getValueAt(i, 2).toString();
                String s = model.getValueAt(i, 3).toString();
                String p = passwords.getOrDefault(u, "");

                pw.println(u + "|" + f + "|" + r + "|" + s + "|" + p);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    // ===== LOAD USERS =====

```

These methods hide user-related logic from the user interface. The interface only calls these methods without knowing how users are loaded, saved, or validated. This makes the system easier to manage and maintain.

4.1.2 Encapsulation

```

import java.io.PrintWriter;

public class UserManager {

    private final Path USERS_FILE = Paths.get("users.txt");
    private final Map<String, String> passwords = new HashMap<>();

    // ===== LOAD USERS =====
    public void loadUsers(DefaultTableModel model) {

```

```

56     }
57
58     // ===== PASSWORD =====
59     public void setPassword(String username, String password) {
60         if (username == null) return;
61         if (password == null) password = "";
62         passwords.put(username, password.trim());
63     }
64
65
66     public boolean checkPassword(String username, String password) {
67         return passwords.containsKey(username)
68             && passwords.get(username).equals(password.trim());
69     }
70
71     // ===== TABLE HELPERS =====
72     public static boolean usernameExists(DefaultTableModel model)

```

The password data is kept private and cannot be accessed directly. Only specific methods are provided to update or verify passwords. This protects sensitive information and prevents direct modification.

4.1.3 Polymorphism

```

private javax.swing.table.DefaultTableModel userModel;
private final java.nio.file.Path USERS_FILE = java.nio.file.Paths.get(
    // helper instance (declared here, created in constructor)
    private UserManager um;

private void setupTableModel() {
    String[] cols = {"Username", "Full Name", "Role", "Status"};
    userModel = new javax.swing.table.DefaultTableModel(cols, 0);
    @Override public boolean isCellEditable(int r, int c) { return false; }
    userTable.setModel(userModel);
    userTable.setRowHeight(26);
}

```

The same JTable component is used to display different user records such as active and inactive users. By changing the table model data, the system reuses the same interface structure to handle different user information. This demonstrates polymorphism through reusable UI components.

4.1.4 Separation of Concerns

UserManagementUI → handles user interface

UserManager → handles user logic

FileHandler → handles file storage

Each class has a clear responsibility. The interface handles user actions, the UserManager processes logic, and file handling manages storage. This separation keeps the system clean and easy to modify.

4.2 Eligibility Check and Enrolment

4.2.1 Abstraction

```
public abstract class Person {

    /** Unique identifier for the person (e.g., student ID) */
    protected String id;

    /** Full name of the person */
    protected String fullName;

    /**
     * Constructor - Initializes person with ID and name
     * Called by subclass constructors using super(id, fullName)
     *
     * @param id Unique identifier
     * @param fullName Full name of the person
     */
    public Person(String id, String fullName) {
        this.id = id;
        this.fullName = fullName;
    }

    /**
     * Gets the person's ID
     * @return The unique identifier
     */
    public String getId() {
        return id;
    }

    /**
     * Gets the person's full name
     * @return The full name
     */
    public String getFullName() {
        return fullName;
    }

    /**
     * ABSTRACT METHOD - Must be implemented by all subclasses
     * @return A string describing the type of person
     */
    public abstract String getPersonType();
}
```

Abstraction is implemented using the abstract class, Person. It defines attributes like id and fullName and getters, while forcing subclasses to implement getPersonType (). Ensuring unnecessary details are hidden and provides a blueprint for different “person types” in the system, while also providing reusable getters.

4.2.2 Inheritance

```
public class EligibilityRecord extends Person implements Eligibility {

    private double cgpa;
    private int failedCourses;

    /** Current eligibility status (ELIGIBLE, INELIGIBLE, or PENDING) */
    private EligibilityStatus eligibilityStatus;

    /** Enrollment approval status: true = Approved, false = Not Approved */
    private boolean enrollment;

    /**
     *
     * INHERITANCE: Calls Person constructor with super() to initialize
     * inherited fields (id, fullName)
     *
     * @param studentId      Unique student identifier
     * @param fullName       Student's full name
     * @param cgpa           Cumulative Grade Point Average
     * @param failedCourses   Number of failed courses
     * @param eligibilityStatus Current eligibility status
     * @param enrollment      Whether enrollment is approved or not
     */
    public EligibilityRecord(String studentId, String fullName, double cgpa, int failedCourses,
        EligibilityStatus eligibilityStatus, boolean enrollment) {

        super(studentId, fullName); // INHERITANCE: Call parent constructor
        this.cgpa = cgpa;
        this.failedCourses = failedCourses;
        this.eligibilityStatus = eligibilityStatus;
        this.enrollment = enrollment;
    }

    /**
     * Gets the student ID (inherited 'id' field from Person)
     * @return The student's unique identifier
     */
    public String getStudentId() {
        return id; // 'id' is inherited from Person (protected access)
    }
}
```

Inheritance is shown when EligibilityRecord extends Person. This allows EligibilityRecord to reuse id and fullName and from Person instead of duplicating code. The constructor uses super(...) to initialize inherited fields.

4.2.3 Encapsulation

```
public double getCgpa() {
    return cgpa;
}

/**
 * Gets the number of failed courses
 * @return Count of failed courses
 */
public int getFailedCourses() {
    return failedCourses;
}

/**
 * Gets the current eligibility status
 * @return EligibilityStatus enum (ELIGIBLE, INELIGIBLE, or PENDING)
 */
public EligibilityStatus getStatus() {
    return eligibilityStatus;
}

/**
 * Checks if enrollment is approved
 * @return true if approved, false if not approved
 */
public boolean isEnrollmentApproved() {
    return enrollment;
}

/**
 * Sets the eligibility status called by EligibilityService.recalc() when applying eligibility rules
 * @param status The new eligibility status
 */
public void setStatus(EligibilityStatus status) {
    this.eligibilityStatus = status;
}

/**
 * Sets the enrollment approval status called when admin approves or denies enrollment
 * @param approved true to approve, false to deny
 */
public void setEnrollmentApproved(boolean approved) {
    this.enrollment = approved;
}
```

Encapsulation is achieved by keeping specific fields private and using controlled access through getters and setters. This prevents other classes from changing values directly and ensures data is updated only through the proper methods, improving data protection and preventing invalid edits.

4.2.4 Interface

```
public interface Eligibility {  
  
    /**  
     * Gets the current eligibility status  
     *  
     * Implementing classes must provide their own logic:  
     * EligibilityRecord: Returns stored eligibilityStatus field  
     * EligibilityStatus enum: Returns itself (this)  
     *  
     * @return eligibility status (ELIGIBLE, INELIGIBLE, or PENDING)  
     */  
    EligibilityStatus getEligibilityStatus();  
    //Provides a human-readable description of the eligibility  
    String describeEligibility();  
}
```

An interface like in this case defines a contract. Any class that implements Eligibility must use `getEligibilityStatus()` and `describeEligibility()`. This ensures consistent behavior across different implementations, it also provides a readable explanation message.

4.2.5 Polymorphism

```
public enum EligibilityStatus implements Eligibility {

    /**
     * ELIGIBLE status - Student meets all progression criteria
     * Criteria: CGPA >= 2.0 AND failed courses <= 3
     */
    ELIGIBLE {
        @Override
        public String describeEligibility() {
            return "The student meets the academic requirements for progression.";
        }
    },

    /**
     * INELIGIBLE status - Student does NOT meet progression criteria
     * Reason: CGPA < 2.0 OR failed courses > 3
     */
    INELIGIBLE {
        @Override
        public String describeEligibility() {
            return "The student does not currently meet the academic requirements.";
        }
    },

    /**
     * PENDING status - Student's eligibility is still being reviewed
     * Used for new students or those awaiting grade confirmation
     */
    PENDING {
        @Override
        public String describeEligibility() {
            return "The student's eligibility is still under review (PENDING).";
        }
    };

    /**
     * Returns this enum constant as the eligibility status
     * Required by the Eligibility interface
     *
     * @return This enum constant (self-reference)
     */
    @Override
    public EligibilityStatus getEligibilityStatus() {
        return this;
    }
}
```

```
// POLYMORPHISM IN ACTION:
// Same interface type, different implementations
Eligibility eRecord = selectedRecord; // EligibilityRecord implementation
Eligibility eStatus = selectedRecord.getStatus(); // EligibilityStatus implementation

// Show detailed info using describeEligibility() from both implementations
String detailsMessage = "Record View:\n" + eRecord.describeEligibility() +
    "\n\nStatus View:\n" + eStatus.describeEligibility();
```

Polymorphism is demonstrated because describeEligibility() behaves differently depending on the enum value (ELIGIBLE vs INELIGIBLE). The same method call makes different results, making the program more flexible. It is also shown through an interface where both EligibilityRecord and EligibilityStatus implement the Eligibility interface.

4.3 Academic Performance Reporting

Academic Performance Reporting module shows good usage of the concept of object-oriented programming, especially the encapsulation and abstraction which lead to improved protection of data, clarity of the system as well as maintainability. Encapsulation and abstraction are the main features of the system. To defend student and course data, the encapsulation concept is used by using the private attributes as well as special getter functions. The property of abstraction is achieved through the separation of file processing and PDF creation into different classes, which then enables the graphical user interface to work through high-level means only. Such guidelines will lead to better data security and less complexity of the system as well as overall increase in maintainability of the Academic Performance Reporting module.

4.3.1 Encapsulation

The process of encapsulation is achieved by stating that all the class attributes are private and can be accessed only using the public getter-methods, which allow to ensure that the course result data are not directly modified, and that the information on academics is controlled.

```
1  package edu.apu.academicperformancereporting.model;
2
3  public class CourseResult {
4
5      // Encapsulation: fields are private
6
7      private final String courseCode;
8      private final String courseTitle;
9      private final int semester;
10     private final int creditHours;
11     private final String grade;
12     private final double gradePoint;
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28     // Access via public getters only
29
30     public int getCreditHours() {
31         return creditHours;
32     }
33
34     public String getGrade() {
35         return grade;
36     }
37
38     public double getGradePoint() {
39         return gradePoint;
40     }
41 }
42
43
44
45
46
47
48
49
50
51
52
```

Such implementation ensures:

- Data hiding and protection
- Access to course results that is read-only.
- Secure and dependable utilization of course information in calculating GPA/CGPA and generating reports.

Encapsulation thus improves the security, maintainability and correctness of the code which are essential in a system of academic reporting.

4.3.2 Abstraction

Abstraction is done by hiding complex implementation details and revealing only the high-level operations to the rest of the system. Abstraction and Academic Performance Reporting Module Abstraction The GUI can create reports and load data in the Academic Performance Reporting module without understanding how the underlying file-reading mechanisms and PDF generation works.

```
// ===== LOAD STUDENTS =====
public static List<Student> loadStudents(String fileName) {

    List<Student> students = new ArrayList<>();

    try (BufferedReader br = getReader(fileName)) {

        String line;
        while ((line = br.readLine()) != null) {

            // Format:
            // ID, Name, Gender, Year
            String[] p = line.split(",");

            String id    = p[0].trim();
            String name  = p[1].trim();
            String gender = p[2].trim();
            String year  = p[3].trim();

            students.add(new Student(id, name, gender, year));

        }

    } catch (Exception e) {
        e.printStackTrace();
    }

    return students;
}
```

Namely, this abstraction consists of the whole process of reading the file: the GUI calls loadStudents and is provided with a list of Student objects, not caring about how the file is opened or read, and then parsing. As a result the GUI logic is simplified and low level I/O

actions are obfuscated.

```
96 private static BufferedReader getReader(String resourceName) throws IOException {
97
98     ClassLoader cl = DataLoader.class.getClassLoader();
99
100     // IMPORTANT:
101     // your files are inside: src/main/resources/resources/
102     InputStream in = cl.getResourceAsStream(resourceName);
103
104     if (in == null) {
105         throw new FileNotFoundException("Resource not found: " + resourceName);
106     }
107
108     return new BufferedReader(new InputStreamReader(in));
109 }
110 }
111
```

In the same vein, abstraction is in charge of access to resource files in the project structure; other techniques do not need to decide whether the files are to be found in a directory, or a resources folder or the classpath since they only request a reader.

Abstraction in Academic performance report is reached through:

- Separating file access logic and the GUI (DataLoader).
- Hiding report formatting and generation (PDFReport)

4.4 Course Recovery Plan

4.4.1 Abstraction

Class: AbstractDataService

```
// Abstract class - base for all data services
public abstract class AbstractDataService {
    // Abstract method - subclasses must implement
    public abstract void loadData();

    // Common method for file reading
    protected String readFile(String filename) {
        try {
            java.nio.file.Path path = java.nio.file.Paths.get(filename);
            return new String(java.nio.file.Files.readAllBytes(path));
        } catch (IOException e) {
            System.out.println("File Error: " + filename);
            return "";
        }
    }
}
```

The **AbstractDataService** serves as a foundational abstract base class designed to standardize data handling across the application. It enforces a structural contract through the abstract method `loadData()`, compelling all subclasses to implement their specific data loading logic. To reduce code duplication, it also includes a concrete, protected utility method, `readFile(String filename)`. This method leverages Java's NIO package to read file content into a string, incorporating a try-catch block to safely handle `IOExceptions` and return an empty string if a read error occurs.

Class: FailedComponentService

```
public class FailedComponentService extends AbstractDataService {
    private final Map<String, List<FailedCourse>> studentFailuresMap = new HashMap<>();

    public FailedComponentService() {
        loadData();
    }

    // @Override - Implemented from AbstractDataService
    @Override
    public final void loadData() {
        try (BufferedReader br = new BufferedReader(new FileReader("failedcomponents.txt"))) {
            String line;
            br.readLine(); // Skip header

            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                if (parts.length >= 5) {
                    String studentID = parts[0].trim();
                    FailedCourse course = new FailedCourse(
                        parts[1].trim(), parts[2].trim(), parts[3].trim(), parts[4].trim()
                    );
                    studentFailuresMap.computeIfAbsent(studentID, k -> new ArrayList<>()).add(course);
                }
            }
        } catch (IOException e) {
            System.out.println("Error loading failedcomponents.txt");
        }
    }
}
```

The **FailedComponentService** is a concrete implementation that extends the **AbstractDataService** class to manage academic failure records. It explicitly **overrides the loadData() method** to read and parse data from the **failedcomponents.txt** file. Inside this method, the service reads the file line-by-line, splitting comma-separated values to instantiate **FailedCourse** objects. These objects are then organized into a **Map<String, List<FailedCourse>>**, enabling the system to store and retrieve a list of multiple failed components associated with a single student ID.

Class: StudentService

```
public final class StudentService extends AbstractDataService {
    private final Map<String, RecoveryStudent> studentMap = new HashMap<>();

    public StudentService() {
        loadData();
    }

    // @Override - Implemented from AbstractDataService
    @Override
    public void loadData() {
        try (BufferedReader br = new BufferedReader(new FileReader("students.txt"))) {
            String line;
            br.readLine(); // Skip header

            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                if (parts.length >= 4) {
                    String id = parts[0].trim();
                    RecoveryStudent student = new RecoveryStudent(id, parts[1].trim(), parts[2].trim(), parts[3].trim());
                    studentMap.put(id, student);
                }
            }
        } catch (IOException e) {
            System.out.println("Error loading students.txt");
        }
    }
}
```

The StudentService is a final class that also extends AbstractDataService to handle core student profile information. Like its counterpart, it **overrides the loadData() method** to execute specific file-reading logic, targeting the students.txt source file. The method parses student details (such as ID and name) to create RecoveryStudent objects, which are stored in a Map<String, RecoveryStudent> for efficient lookup by student ID.

4.4.2 Encapsulation

Class: FailedCourse

```
public class FailedCourse {  
    private final String courseID;  
    private final String courseName;  
    private final String grade;  
    private final String failedComponent;  
  
    // Constructor  
    public FailedCourse(String courseID, String courseName, String grade, String failedComponent) {  
        this.courseID = courseID;  
        this.courseName = courseName;  
        this.grade = grade;  
        this.failedComponent = failedComponent;  
    }  
  
    // Getters  
    public String getCourseID() { return courseID; }  
    public String getCourseName() { return courseName; }  
    public String getGrade() { return grade; }  
    public String getFailedComponent() { return failedComponent; }  
}
```

The FailedCourse class strictly adheres to the principles of encapsulation to ensure data integrity. Its internal state is protected by declaring all fields—courseID, courseName, grade, and failedComponent—as **private** and **final**. This restricts direct access and ensures immutability once the object is initialized via its constructor. Controlled access to these values is provided exclusively through public **getter methods** (e.g., getCourseID(), getGrade()), effectively hiding the implementation details while allowing safe, read-only retrieval of data.

Class: RecoveryStudent

```
public class RecoveryStudent {  
    private final String studentID;  
    private final String firstName;  
    private final String lastName;  
    private final String major;  
  
    // Constructor  
    public RecoveryStudent(String studentID, String firstName, String lastName, String major) {  
        this.studentID = studentID;  
        this.firstName = firstName;  
        this.lastName = lastName;  
        this.major = major;  
    }  
  
    // Getters  
    public String getStudentID() { return studentID; }  
    public String getFullName() { return firstName + " " + lastName; }  
    public String getMajor() { return major; }  
}
```

The `RecoveryStudent` class effectively implements encapsulation to safeguard student identity data. It isolates internal state by declaring all fields—`studentID`, `firstName`, `lastName`, and `major`—as **private** and **final**. This design ensures that once an instance is created via the constructor, its data remains immutable and cannot be modified externally. Access is strictly controlled through public getter methods; notably, the `getFullName()` method encapsulates a derived property, concatenating `firstName` and `lastName` to return a single string, thereby hiding the internal storage structure from the caller.

4.5 Email Notification Service

4.5.1 Encapsulation

```
private String studentName;  
private String studentID;  
private String recipientEmail;  
private String type;  
private String timestamp;
```

The log details are kept internally and are not changed directly by other parts of the system. This helps keep the recorded information accurate and avoids accidental or unintended changes..

```
public EmailLog(String studentName, String studentID, String recipientEmail, String type, String timestamp) {  
    this.studentName = studentName;  
    this.studentID = studentID;
```

The log data stays inside the system and cannot be changed directly, which helps keep everything correct and avoids mistakes.

```
// ENCAPSULATION: getters only (no modification)  
public String getStudentName() { return studentName; }  
public String getStudentID() { return studentID; }  
public String getRecipientEmail() { return recipientEmail; }  
public String getType() { return type; }  
public String getTimestamp() { return timestamp; }
```

These getters allow read-only access to private data. They provide controlled data access without allowing modification.

```
private final String senderEmail;  
private final String senderPassword;
```

The email details are kept hidden and can't be accessed from outside. Once they are set, they stay the same, which helps keep the information secure and unchanged.

```
public EmailSender(String email, String password) {  
    this.senderEmail = email;  
    this.senderPassword = password;  
}
```

This constructor sets up the email details when the object is created, making sure everything needed is ready before it is used.

```
private static final String LOG_FILE = "sent_email_log.txt";
```

The log file location is kept inside the system and can't be changed from outside, so all records are saved in one fixed place.

```
try (FileWriter fw = new FileWriter(LOG_FILE, true)) {
```

This code ensures file handling is fully managed within the class. Resource management and file

4.5.2 Inheritance

```
public class MilestoneReminderNotification extends Notification {
```

Extends

This allows the notification to use the same data and functions as other notifications, which avoids repeating code and keeps everything structured the same way.

```
    public MilestoneReminderNotification(String studentName, String studentID, String module) {  
        super(studentName, studentID, module);  
    }
```

The constructor uses super to initialize data defined in the parent Notification class. This ensures the inherited fields are correctly set when the object is created.

```
public class PasswordRecoveryNotification extends Notification {
```

This line demonstrates **inheritance** by allowing PasswordRecoveryNotification to inherit fields and methods from the Notification class. It reuses common notification data such as student name and ID. This avoids code duplication and ensures all notifications follow the same structure.

```
    public PasswordRecoveryNotification(String studentName, String studentID) {  
        super(studentName, studentID, null);  
    }
```

This sets the shared values using the parent setup, making sure the inherited information is ready when the object is created.

```
public class RecoveryAssignedNotification extends Notification {  
    public RecoveryAssignedNotification(String name, String id, String module) {  
        super(name, id, module);  
    }
```

This line demonstrates **inheritance** by extending the Notification class. It allows the subclass to reuse common notification attributes and methods. This ensures consistency across all notification types and avoids duplicate code and also constructor uses the super keyword to initialize fields

defined in the parent Notification class. It ensures the inherited data is properly set when the object is created.

```
// INHERITANCE + POLYMORPHISM
public class SentEmailLog extends EmailLog {

    public SentEmailLog(String studentName, String studentID, String recipientEmail, String type, String timestamp) {
        super(studentName, studentID, recipientEmail, type, timestamp);
    }
}
```

SentEmailLog demonstrates inheritance by extending the abstract EmailLog class using extends EmailLog, which allows SentEmailLog to reuse all properties and methods defined in the parent class. Common log attributes such as student details and email information are inherited to maintain a consistent log structure. The constructor uses the super(...) keyword to initialize fields declared in EmailLog. This ensures all inherited data is correctly set when a SentEmailLog object is created.

```
public class UserAccountManagementNotification extends Notification {

    public UserAccountManagementNotification(String studentName, String studentID) {
        super(studentName, studentID, null);
    }
}
```

UserAccountManagementNotification demonstrates inheritance by extending the Notification class using extends Notification, which allows the class to reuse common notification fields and methods. The constructor uses the super(...) keyword to initialize fields declared in the parent class, ensuring inherited data is correctly set when the object is created. This reuse of parent class functionality maintains consistency across notification types and avoids code duplication.

4.5.3 Polymorphism

```
// POLYMORPHISM: all subclasses MUST implement this
public abstract String formatLogEntry();
```

This enforces a common interface for log formatting. Subclasses must implement it, allowing different behaviors to be executed at runtime.

```

    Session session = Session.getInstance(props, new Authenticator() {
        @Override
        protected PasswordAuthentication getPasswordAuthentication() {
            return new PasswordAuthentication(senderEmail, senderPassword);
        }
    });

```

This code overrides the authentication behavior using an anonymous subclass of `Authenticator`. Java executes the overridden method at runtime to supply the correct credentials.

```
fw.write(log.formatLogEntry() + "\n");
```

The system uses the `EmailLog` type, but the actual formatting comes from the specific log being used at runtime, allowing different log formats to work automatically.

```

@Override
public String generateMessage() {

```

It overrides `generateMessage()` from the parent class. When called through a `Notification` reference, Java executes this subclass implementation at runtime.

```

@Override
public String generateMessage() {
    return "Subject: Password & Recovery Information\n\n"
        + "Dear " + getStudentName() + " (" + getStudentID() + "),\n\n"
        + "Your password recovery request has been processed.\n"
        + "If you initiated this request, please follow the recommended security steps "
        + "to update your password.\n\n"
        + "If this was NOT you, please reset your password immediately.\n\n"
        + "Regards,\nCourse Administrator";
}

```

Override method to demonstrate polymorphism by overriding the abstract `generateMessage()` method from the parent class. When called through a `Notification` reference, Java executes this subclass's implementation at runtime. This allows different notification types to produce different messages using the same method name.

```

@Override
public String generateMessage() {
    return "Subject: Course Recovery Assigned\n\n"
        + "Dear " + getStudentName() + " (" + getStudentID() + "),\n\n"
        + "You have been assigned a recovery plan for the module: " + getModuleName() + ".\n\n"
        + "Please review the tasks and deadlines.\n\n"
        + "Regards,\nCourse Administrator";
}

```

Polymorphism by overriding the abstract `generateMessage()` method from the parent class. When accessed through a `Notification` reference, Java executes this subclass's implementation at runtime. This allows different notification types to generate different messages using the same method.

```

@Override
public String formatLogEntry() {
    return getTimestamp() + " | " +
        getStudentID() + " | " +
        getStudentName() + " | " +
        getRecipientEmail() + " | " +
        getType();
}

```

This method demonstrates polymorphism by overriding the abstract `formatLogEntry()` method from `EmailLog`. When called using an `EmailLog` reference, Java executes this subclass implementation at runtime, allowing different log formats to be handled dynamically.

```

@Override
public String generateMessage() {
    return "Subject: User Account Management Update\n\n"
        + "Dear " + getStudentName() + " (" + getStudentID() + "),\n\n"
        + "This email is to notify you that updates have been made to your user account at APU.\n\n"
        + "These changes may include profile updates, login activity, or system access permissions.\n\n"
        + "If you did not request these changes, please contact support immediately.\n\n"
        + "Regards,\n"
        + "Course Administrator";
}

```

It demonstrates polymorphism by overriding the abstract `generateMessage()` method from the parent class. When called using a `Notification` reference, Java executes this subclass implementation at runtime to generate the correct message.

4.5.4 Abstraction

```
// ABSTRACTION
public abstract class EmailLog {
```

Abstract class defines the structure of an email log without allowing direct object creation. It hides implementation details and forces subclasses to complete the behavior.

```
public boolean sendEmail(String toEmail, String subject, String body) {
```

This exposes a simple interface for sending emails while hiding all SMTP and authentication logic. The caller only provides required inputs without knowing how the email is sent.

```
    Transport.send(message);
    return true;
```

This line delegates the email-sending operation to the Jakarta Mail API. The internal delivery process is hidden, allowing the system to send emails using a single method call.

```
    } catch (AddressException ae) {
        // Gmail says invalid email format or doesn't exist
        throw new RuntimeException("INVALID_EMAIL");
    } catch (MessagingException me) {
        // Other Gmail failures
        throw new RuntimeException("EMAIL_SEND_FAILED");
    } catch (Exception e) {
        throw new RuntimeException("UNKNOWN_ERROR");
    }
}
```

These catch blocks abstract different email-sending failures into simple runtime error codes. This allows the rest of the system to handle errors without understanding low-level email exceptions.

```
public static void writeLog(EmailLog log) {
```

It provides a simple interface for writing logs without exposing file-handling details. Other classes only call this method and do not manage file I/O directly.

```
} catch (IOException e) {  
    System.out.println("LOG ERROR: " + e.getMessage());  
}
```

This block abstracts low-level file errors into a simple log message. The calling code is not required to handle file system exceptions.

```
return "Subject: Milestone Reminder\n\n"  
    + "Dear " + getStudentName() + " (" + getStudentID() + "),\n\n"  
    + "This is a reminder for your upcoming milestone in "  
    + getModuleName() + ".\n\n"  
    + "Please ensure you are following the recovery plan timeline.\n\n"  
    + "Regards,\nCourse Administrator";
```

This implementation provides the specific message content while following the abstract method contract. The caller does not need to know how the message is constructed, only that it can be generated.

```

public Notification(String studentName, String studentID, String moduleName) {
    this.studentName = studentName;
    this.studentID = studentID;
    this.moduleName = moduleName;
}

public String getStudentName() {
    return studentName;
}

public String getStudentID() {
    return studentID;
}

public String getModuleName() {
    return moduleName;
}

public abstract String generateMessage();
}

```

A shared structure is set up so all notifications follow the same format, while direct creation is restricted. The required details are set when the object is created and accessed in a controlled way. Each notification then provides its own message content. This keeps everything consistent while still allowing different notification behavior.

```

public static Notification createNotification(String type, String name, String id, String module) {
    switch (type) {
        case "Course Recovery Plan Assigned":
            return new RecoveryAssignedNotification(name, id, module);

        case "Milestone Reminder":
            return new MilestoneReminderNotification(name, id, module);

        case "Student Account Management":
            return new UserAccountManagementNotification(name, id);

        case "Password & Recovery Management":
            return new PasswordRecoveryNotification(name, id);

        default:
            throw new IllegalArgumentException("Unknown notification type: " + type);
    }
}
}

```

All notification objects are created in one place, so the rest of the system doesn't need to know how they are made. The type value decides which notification is returned, while everything is

treated as the same base type. This keeps the code easier to manage and makes adding new notification types straightforward.

CHAPTER 5: LIMITATIONS

The **Course Recovery System (CRS)** operates within defined technical, operational, and functional boundaries that reflect its scope as an academic coursework project. The following limitations have been identified and documented:

1. File-Based Data Storage Only

The system relies exclusively on text or binary files for data persistence and manipulation. This approach lacks the robustness, scalability, and query flexibility of a relational database management system (RDBMS). File-based storage may encounter performance degradation with large datasets and does not support concurrent data access or advanced data retrieval operations.

2. Single-User Session Architecture

The system is designed to support only one active user session at a time. Concurrent multi-user access, session management, and simultaneous operations are not supported. This limitation restricts the system's usability in high-traffic institutional environments where multiple administrators may need simultaneous access.

3. Limited Data Query and Reporting Capabilities

File-based operations restrict advanced querying and analytics. The system cannot perform complex SQL-like queries or generate sophisticated analytical reports beyond the predefined academic performance report structure.

4. Manual Data Entry Requirement

All user accounts, courses, and initial student data must be manually entered through the GUI. The system does not support automated data import from CSV files or institutional data feeds, requiring significant administrative effort during initial setup.

5. Limited User Role Flexibility

The system supports only two predefined user roles: Academic Officer and Course Administrator. Role-based permissions are fixed and cannot be customized. Intermediate or custom roles cannot be created to accommodate institutional variations.

6. Scalability Constraints

Performance may degrade significantly when managing large numbers of students (500+),

courses (100+), or recovery plans. File I/O operations can become bottlenecks with extensive data volumes.

7. Limited Reporting Format Options

Academic performance reports are generated exclusively in PDF format. Alternative formats such as Excel, JSON, or XML are not supported.

8. Basic Authentication Only

The system implements basic username/password authentication without advanced security measures such as two-factor authentication (2FA), OAuth integration, or password encryption policies beyond what is standard.

9. Limited Data Protection

Stored passwords and sensitive data rely on basic encoding mechanisms. Advanced encryption standards or secure key management are not implemented.

10. No System Help Documentation

The system does not include integrated help menus, tooltips, or context-sensitive guidance. Users must rely on external documentation or training to navigate system features and complete operations.

11. No Backup and Recovery Mechanisms

The system does not provide automated backup functionality or disaster recovery procedures. Data loss due to file corruption or accidental deletion cannot be automatically recovered without manual file restoration.

CONCLUSION

The Course Recovery System (CRS) was an effective system that was proposed and developed as a desktop-based Java application to aid in the management of academic recovery within institutions of learning. The system deals with the main problems which are dealing with unsuccessful course elements, assessment of progression eligibility, producing academic performance reports, scheme of recovery plans and automatic email alerts. CRS enhances the efficiency, accuracy, and transparency in the workflow of academic recovery by combining these functions into a combined platform. One of the aims of the project was to illustrate the successful application of Object-Oriented Programming (OOP) principles. The concept of abstraction was used to conceal the complex process like email delivery, logging, and processing data behind the simple interfaces and abstract classes. The concept of encapsulation guaranteed the integrity and security of the data because it did not allow direct access to sensitive fields and revealed controlled methods. Inheritance allowed reuse and consistency of code between classes related together especially in notification and logging packages, and polymorphism allowed variation in runtime behavior by method overriding and interface implementation. It has a user-friendly GUI, data handling in a structured manner, CGPA calculation, generation of PDF report, the ability to track recovery milestone and real email integration using Java Mail API. Though the system has some limitations such as file-based storage, single user access and simple security measures, these are within the scope of the coursework. On the whole, the CRS exhibits robust implementation of OOP principles and software development patterns, which will offer a sufficient base on further advancement and the application into practice.

REFERENCES

- creately. (2025, October 9). *What is a UML Class Diagram*. Retrieved from creatively:
<https://creately.com/guides/what-is-a-class-diagram/>
- GeeksforGeeks. (2025, May 7). *Association, Composition and Aggregation in Java*. Retrieved from GeeksforGeeks: <https://www.geeksforgeeks.org/java/association-composition-aggregation-java/>
- GeeksforGeeks. (2025, July 23). *Difference Between Abstract Class and Interface in Java*. Retrieved from GeeksforGeeks: <https://www.geeksforgeeks.org/java/difference-between-abstract-class-and-interface-in-java/>
- GeeksforGeeks. (2025, September 24). *Java OOP(Object Oriented Programming) Concepts*. Retrieved from GeeksforGeeks: <https://www.geeksforgeeks.org/java/object-oriented-programming-oops-concept-in-java/>
- iText. (n.d.). *iText 5 Java*. Retrieved from iText: <https://itextpdf.com/resources/api-documentation/itext-5-java>
- JVMHOST. (2012, May 14). *How to send mail with JavaMail (SMTP/SMTPS/SSL)?* Retrieved from JVMHOST: <https://www.jvmhost.com/articles/how-to-send-mail-with-javamail-and-tomcat/>
- Visual Paradigm. (n.d.). *What is Use Case Diagram?* Retrieved from Visual Paradigm: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-use-case-diagram/>
- W3Schools. (2025, December 13). *Java Inheritance*. Retrieved from W3Schools: https://www.w3schools.com/java/java_inheritance.asp
- W3Schools. (n.d.). *Java Abstraction*. Retrieved from W3Schools: https://www.w3schools.com/java/java_abstract.asp

W3Schools. (n.d.). *Java Classes and Objects*. Retrieved from W3Schools:

https://www.w3schools.com/java/java_classes.asp

W3Schools. (n.d.). *Java Encapsulation*. Retrieved from W3Schools:

https://www.w3schools.com/java/java_encapsulation.asp

W3Schools. (n.d.). *Java Polymorphism*. Retrieved from W3Schools:

https://www.w3schools.com/java/java_polymorphism.asp

WORKLOAD MATRIX

No	TP	Name	Component
1	TP084807	AHMED MIRAHUSAIN ALVI	• Course Recovery Plan • Class Diagram • Program Code Compilation
2	TP082459	MOHAMED ABDIFATAH ALI	• Academic Performance Report • Use Case Diagram • Use Case Diagram Description
3	TP085327	SULTAN ABDULLA OMAR TAKRORI	• Email Notification Service • Program Code Compilation • Conclusion
4	TP084630	MUHAMMAD ZIAD	• Eligibility Check & Enrollment • Program Code Compilation • Overall Documentation
5	TP085061	MUHAMMAD TAIMOOR JAMAL	• User Management • Introduction • Login